# 컴퓨터구조

Presented by
오 세 춘

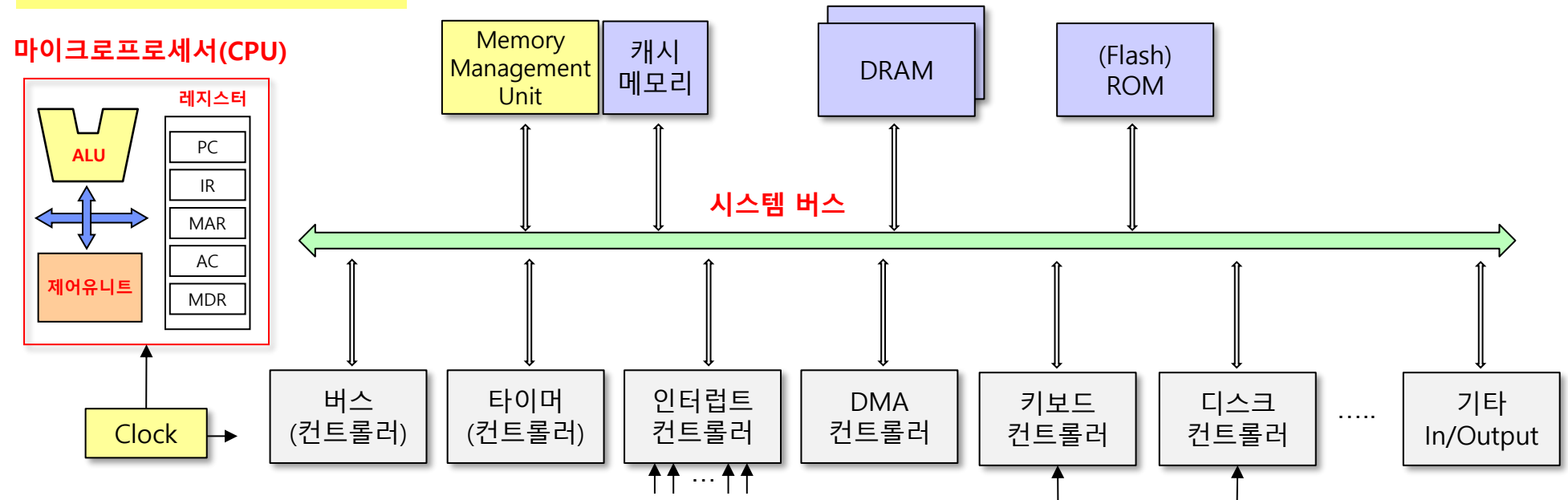
## 제1장 컴퓨터 시스템 개요

# 교재 주요 구성

## ■ 컴퓨터 시스템의 구성 – 교재 및 강의 구성 내용

제1장 : 컴퓨터시스템 개요

제5장 : 기억장치




제2장 : CPU의 구조와 기능

제3장 : 컴퓨터 산술과 논리 연산(ALU)

제4장 : 제어 유니트

제7장 : 시스템 버스와 IO 및 인터럽트

추가: 가상메모리

제6장 : 보조기억장치

제8장 : 고성능 컴퓨터시스템 구조

# 배울 내용



## ■ 목차

✓ 1.1 컴퓨터의 기본 구조

✓ 1.2 정보의 표현과 저장

✓ 1.3 시스템의 구성

✓ 1.4 컴퓨터구조의 발전과정

✓ 1.5 컴퓨터 성능 (교재에 없음)

# 컴퓨터란 ?



## ■ 컴퓨터의 어원

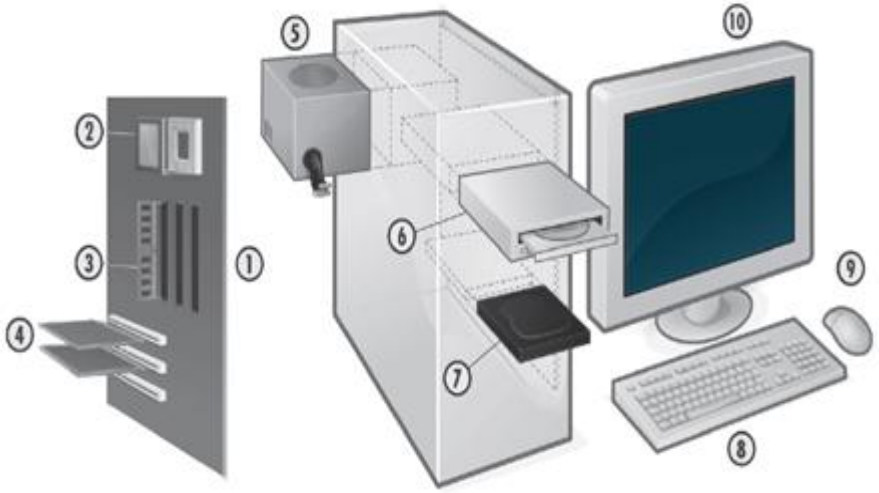
- ✓ 컴퓨터(Computer)라는 명칭은 계산한다는 뜻의 'Compute'에서 유래
  - ➢ 계산하는 사람 또는 계산하는 기계

## ■ 컴퓨터 시스템의 개념적 구성

- ✓ 하드웨어 + 소프트웨어
- ✓ 펌웨어(Firmware)




## ■ 컴퓨터 하드웨어의 주요 요소들



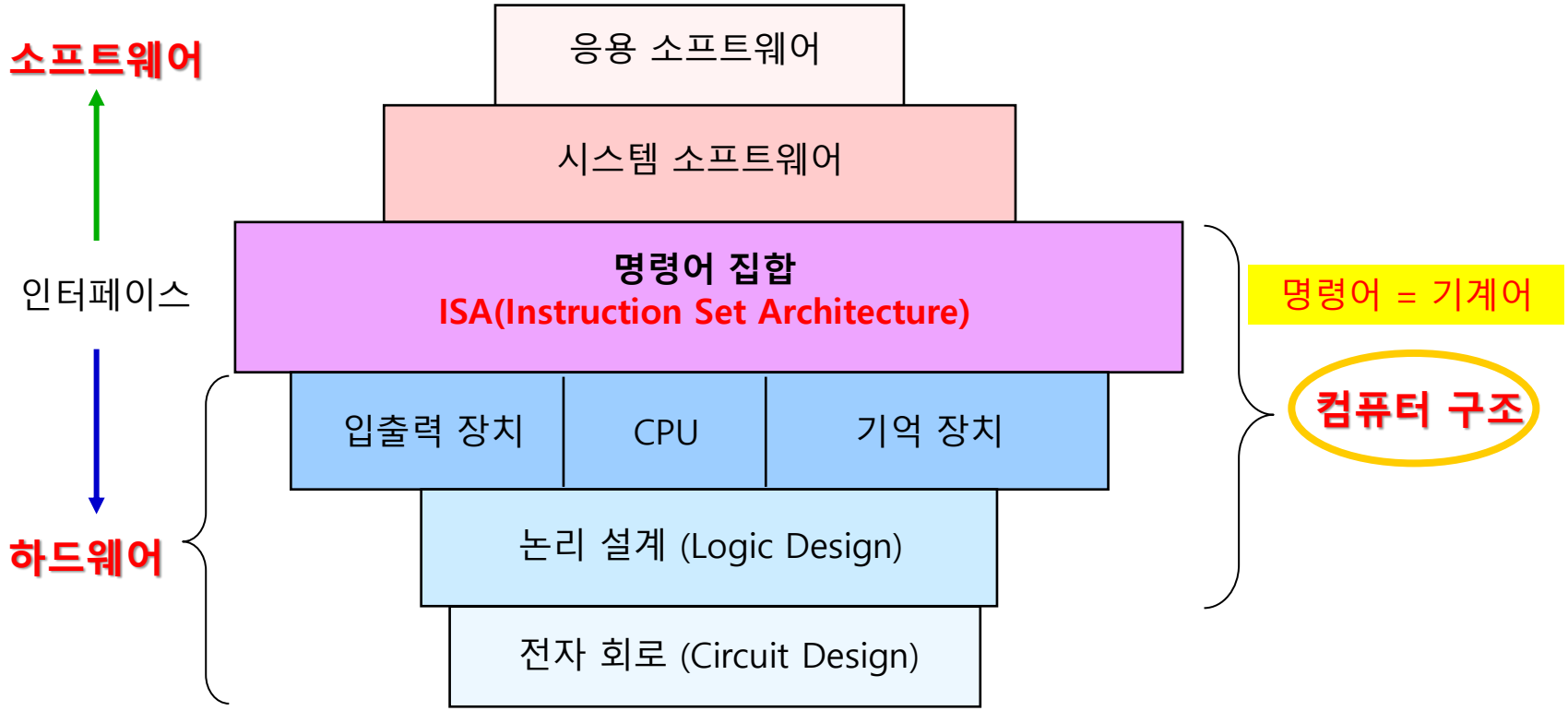

① 메인 보드(main board)
② CPU 및 GPU 칩
③ 주기억장치 모듈
④ 확장 보드: 사운드 카드 등
⑤ 전원공급장치(power supply)
⑥ 광 저장장치: CD-ROM, DVD
⑦ 하드 디스크, SSD
⑧ 키보드
⑨ 마우스
⑩ 디스플레이 모니터

# 1. 컴퓨터 아키텍처



## ■ 컴퓨터 아키텍처(Architecture = 구조)의 정의

✓ 하드웨어와 소프트웨어를 포함하는 시스템 전체의 설계방식




**컴퓨터 구조 ≒ 스마트폰 구조 ≒ 제어시스템 구조**

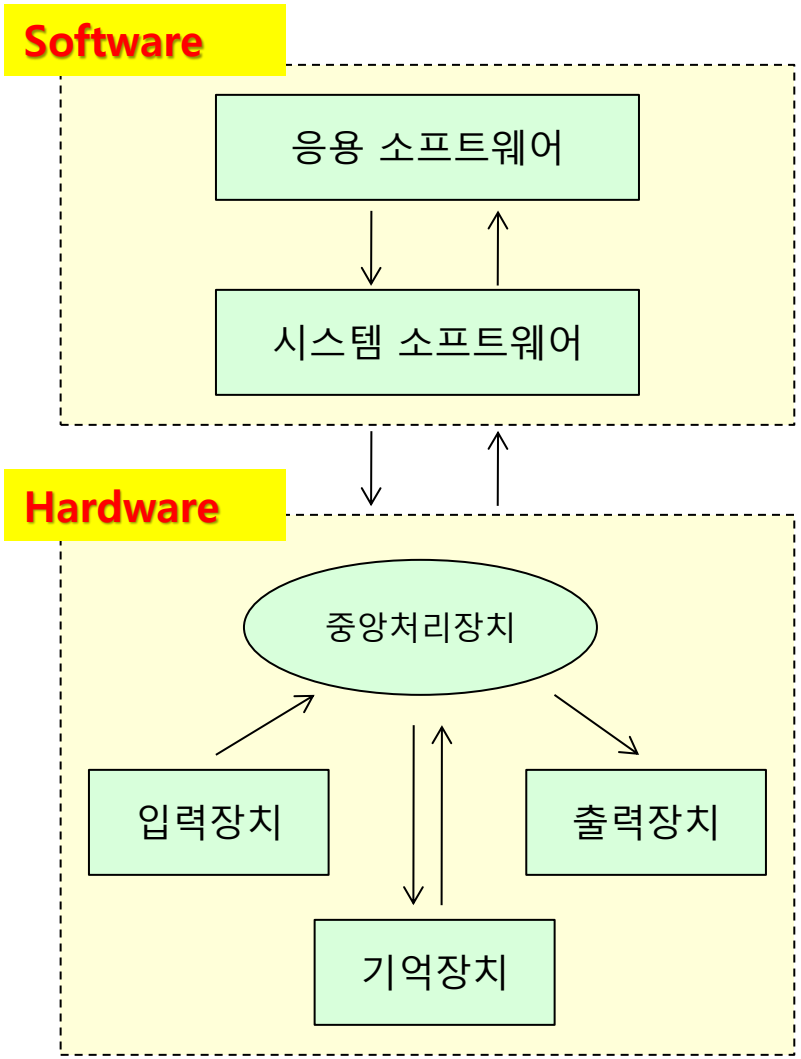
# 1. 컴퓨터 아키텍처



p14~p16

## ■ 하드웨어 (Hardware)

1) 중앙처리장치 (CPU: Central Processing Unit)
   - ➢ '프로세서', '처리기' 라고도 함
   - ➢ **모든 컴퓨터의 작동이 CPU(두뇌)의 제어**를 받음
   - ➢ 마이크로프로세서는 CPU를 집적회로 형태로 만든 물리적 구성 요소
2) 입출력장치
   - ➢ 키보드, 마우스, 스캐너, 디지타이저, 터치스크린, 조이스틱, CCD 등
   - ➢ 모니터(LCD,OLED), 프린터, 스피커, 플로터, Haptic, HMD, HUD 등
3) 기억장치
   - ① 주기억장치: RAM, ROM
   - ② 보조기억장치: HDD, SSD, USB 메모리 등등



# 1. 컴퓨터 아키텍처



p14~p16

## ■ 소프트웨어 (Software)

### 1) 응용 소프트웨어

- ➢ 최종 사용자를 위해 개발된 S/W
- ➢ 주로 High-level 언어로 작성

### 2) 시스템 소프트웨어

- ➢ 컴퓨터 하드웨어를 제어, 관리하는 가장 기본적인 S/W

① 시스템 유틸리티

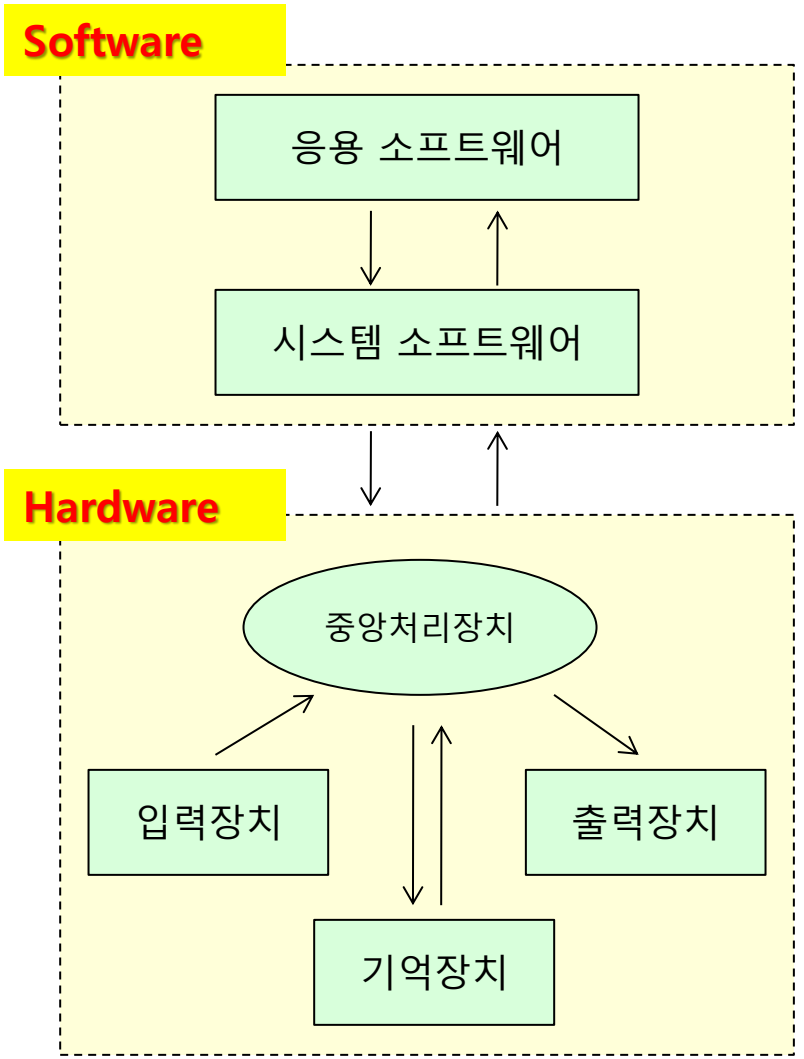
- ❖ 컴파일러, 어셈블러, 링커, 에디터 등

② 운영체제

- ❖ 프로세스 관리, 메모리 관리, 주변장치 관리, 파일관리, Device-driver 등

③ Firmware (펌웨어)

- ❖ 일반적으로 ROM에 저장된 하드웨어를 제어하는 프로그램을 의미
- ❖ 예: PC에서의 ROM BIOS = POST + 초기화 + 부트로더



# 1. 컴퓨터 아키텍처

## ■ 시스템 소프트웨어 - 펌웨어(Firmware)

ROM에 저장되어 있는
(시스템 또는 응용) 프로그램

✓ 펌웨어는 일반적으로 롬(ROM)에 저장된 하드웨어를 제어하는데 사용되는 프로그램을 의미

✓ CPU 내부 메모리 또는 CPU 외부의 ROM(또는 플래시 메모리)과 같은 비휘발성 기억장치에 있음

✓ 배경

① 소프트웨어를 통해 전달된 정보(기계어, 명령어)를 받아들인 하드웨어는 내부의 논리 회로를 거쳐 사용자가 원하는 계산 결과의 출력이나 특정 기기의 동작 등을 실행한다

② 그런데 컴퓨터 시스템의 활용 범위가 넓어지고, 소프트웨어에서 전달되는 정보 역시 방대해지다 보니 하드웨어 내 제한된 종류의 논리 회로만으로는 이러한 다양한 상황에 모두 대응하기가 어렵게 되었다

③ 물론, 새로운 소프트웨어가 등장할 때마다 그에 해당하는 기능을 갖춘 논리 회로를 추가한 하드웨어를 새로 만들 수도 있겠지만, 이렇게 하면 비용이나 시간 면에서 큰 낭비가 아닐 수 없다.

④ 그래서 컴퓨터 개발자들은 하드웨어 내부의 제어 유니트 부분에 저장공간을 만들어, 그곳에 논리 회로의 기능을 보강하거나 대신할 수 있는 프로그램을 넣을 수 있게 하였다. 이것이 바로 '펌웨어(Firmware)'이다. 추가로 외부 기억장치에도 존재할 수 있다.

⑤ 따라서, 같은 종류의 하드웨어라고 해도 펌웨어가 달라지면 기능이나 성능, 혹은 사용하는 소프트웨어의 종류가 달라질 수 있다. 펌웨어는 프로그램의 형태를 갖추고 있으므로 기능적으로는 소프트웨어에 가깝지만 하드웨어 내부에 위치하며, 사용자가 쉽게 그 내용을 바꿀 수 없으므로 하드웨어적인 특성도 함께 가지고 있다고 할 수 있다

⑥ 근래에는 아두이노, ESP32, ARM 프로세스 등의 MCU 내부 ROM(Flash 메모리)에 포함된 프로그램을 의미하기도 함

# 1. 컴퓨터 아키텍처

## ■ 시스템 소프트웨어 - 펌웨어(Firmware)

✓ **PC**에서는 외부 ROM용 펌웨어를 ROM **BIOS**(Basic Input Output System)라고 부름

### 1) 부팅

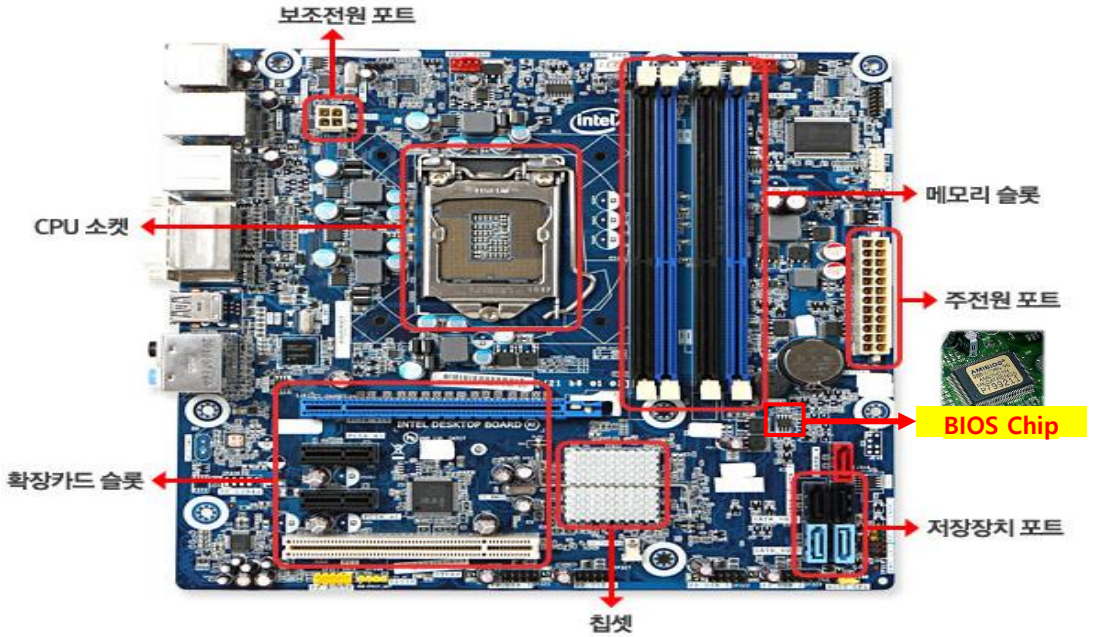
➢ 컴퓨터 ON 시에

① POST(Power-On Self Test) 실시 → 스피커, LED(7-segment) 출력

② 하드웨어를 초기화

③ Booting을 실행 (운영체제 중 선택된 하나를 Loading)



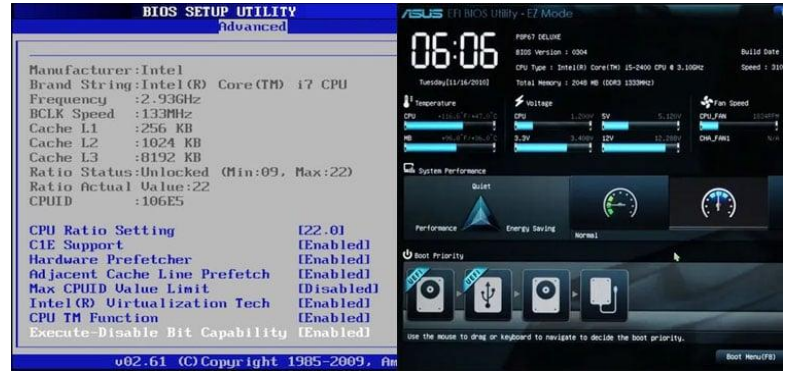

### 2) PC의 각종 초기치 설정 (setup)

### 3) Interrupt Handler 역할

① 하드웨어 인터럽트 처리

② 프로그래머가 사용할 수 있는 서비스 루틴



# 1. 컴퓨터 아키텍처

## ■ 시스템 소프트웨어 - 펌웨어(Firmware)

✓ 근래의 PC ROM BIOS는 2가지 모드가 존재

### 1) Legacy BIOS

- 레거시 바이오스라고 부르며 PC 초창기부터 계속 사용하는 BIOS 모드 → 거의 사용하지 않음
- POST 실시 → 하드웨어 초기화 → 부팅시 읽어 들일 저장장치 순서 결정 → Booting 실행
- **MBR 파티션**을 사용하기 때문에 2.2TB 이상의 저장장치를 인식하지 못하며, MBR 파티션 내의 부트 섹터를 거친 후 윈도우를 불러와서 부팅 속도가 느림

### 2) UEFI BIOS (unified Extensible Firmware Interface)

- 통일 확장형 펌웨어 인터페이스로써 '유-이-에프-아이' 또는 '유이파이' 라고도 읽음 → 주로 사용하는 방식
- 2006년 이후의 대부분의 메인 보드에서 추가로 지원하는 모드
  - ❖ 기존의 16비트 x86 기반 BIOS의 한계 극복은 물론 새로운 디바이스에 대한 유연한 대처가 목적

① **GPT 파티션** 지원: 2.2TB 이상의 저장장치를 지원 (기존의 MBR 파티션도 사용 가능)

② Fast Boot 기능: 기존 BIOS 코드를 생략하고 Boot Manager를 통해 바로 부팅하여 부팅속도 개선

③ Secure Boot 기능: 정상적으로 인증되지 않은 운영체제 부트로더와 드라이버의 사용을 금지

# 1.1 컴퓨터 기본 구조

p16~p17

## ■ 컴퓨터의 주요 구성 요소

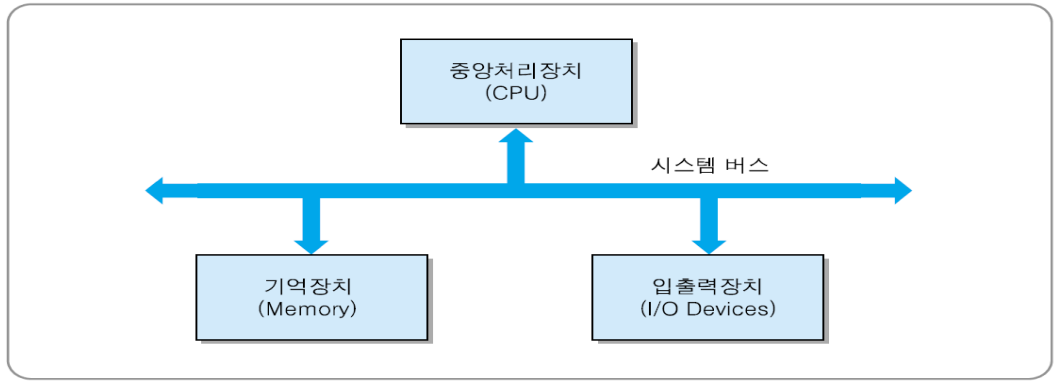
① 중앙처리장치 (CPU)

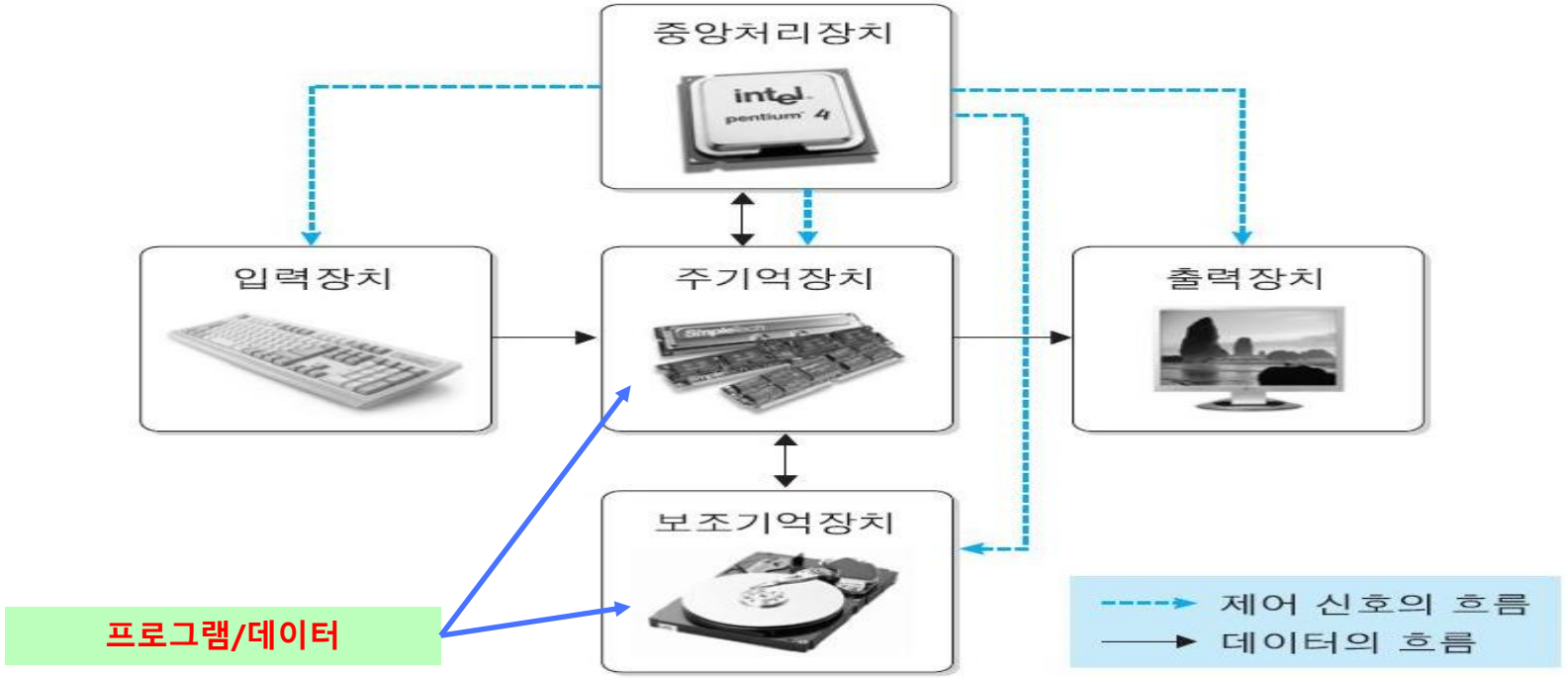
② 입력장치

③ 출력장치

④ 기억장치

⑤ 시스템 버스



# 1.1 컴퓨터 기본 구조

p16~p17

## 1) 중앙처리장치(CPU: Central Processing Unit)

- ✓ 프로세서(Processor), 처리기, 마이크로프로세서 라고도 함
- ✓ MPU(Micro Processor Unit)라고도 부름 ↔ MCU(Micro Controller Unit)
- ✓ '프로그램 실행' 과 '데이터 처리' 라는 중추적인 기능의 수행을 담당하는 요소

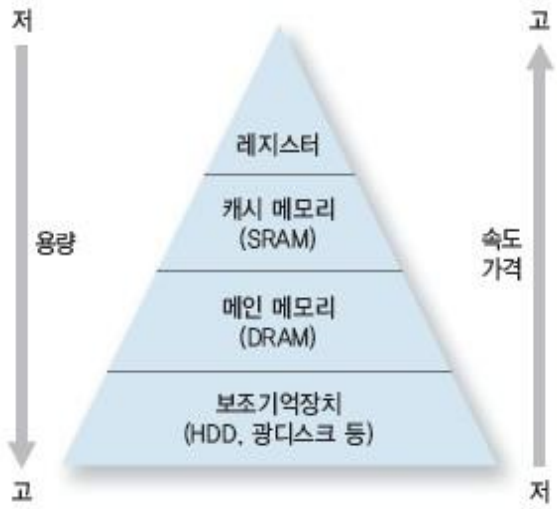
**참고: CPU(Central Processing Unit) 의 어원**
초기에는 하나의 컴퓨터가 여러 개의 유니트(캐비닛)들로 구성되어 있었으며 이중에서 핵심적인 역할을 하는 유니트를 '중앙처리장치' 라고 불렀기 때문에 지금도 사용



## 2) 기억장치

① 주기억장치(Main Memory)

- ➢ 메인 메모리라고도 부름
- ➢ CPU가 시스템 버스를 통해 기억장치의 내부의 모든 주소들을 직접 지정(읽기, 읽기/쓰기) 가능
- ➢ CPU 가까이 위치하며, 고속의 반도체 기억장치 칩들로 구성
- ➢ 가격이 높고 면적을 많이 차지 → 저장 용량의 한계
- ➢ (주로) 영구 저장 능력이 없기 때문에, 일시적 저장장치로만 사용
- ➢ 실행될 프로그램들이 순차적으로 놓여 있다 → **Stored program** (폰 노이만)




< 메모리 계층 구조 >

# 1.1 컴퓨터 기본 구조



p16~p17

## 2) 기억장치 - 계속

② 보조기억장치(auxiliary storage device)

- 2차 기억장치(secondary memory)
- CPU가 저장장치 내부의 모든 주소를 직접적으로 지정하지는 못함
- 저장 밀도가 높고, 비트 당 가격이 낮고 읽기/쓰기 속도가 상대적으로 느림
- 영구 저장 능력을 가진 저장장치: 하드 디스크, 플래시 메모리, SSD, CD-ROM, 등

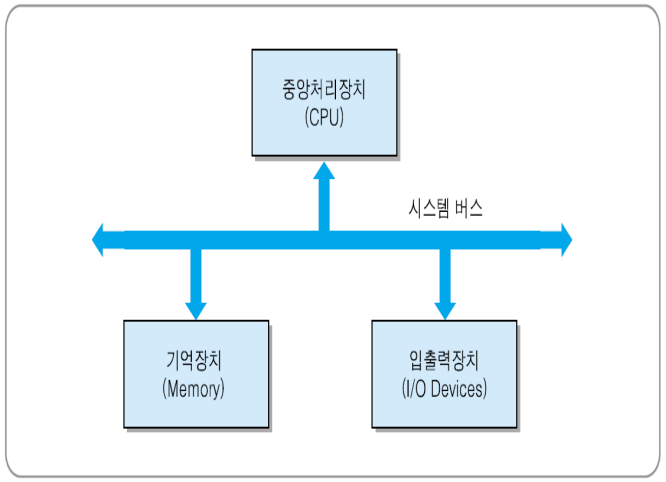
| 커넥터 종류 | 핀수 | 주요 특징 |
|---|---|---|
| USB-A (2.0) | 4 | 가장 기본적인 타입, 하위 호환 |
| USB-A (3.0/3.1) | 9 | 파란색 단자, 초고속 통신 핀 추가 |
| Micro-USB | 5 | 구형 스마트폰/소형 기기, OTG 핀 포함 |
| USB-C | 24 | 최신 표준, 양면 동일, 다기능 지원 |
| SATA 데이터 | 7 | 메인보드와 데이터 전송 |
| SATA 전원 | 15 | 파워 서플라이로부터 전력 공급 |

## 3) 입출력장치(I/O = Input/Output)

- ✓ 사용자와 컴퓨터 간의 대화를 위한 입력 및 출력장치로 종류와 속도가 다양
- ✓ 주변장치(peripheral device)라고도 부름
- ✓ CPU는 해당 장치 제어기(device controller)를 통하여 동작을 제어

## 4) 시스템 버스(System bus)

- ✓ CPU와 CPU 외부장치와의 연결 통로로서 전기적 신호선의 묶음을 의미
  - 보통 수십 개 ~ 수백 개의 신호선들을 하나의 명칭으로 부를 때 사용

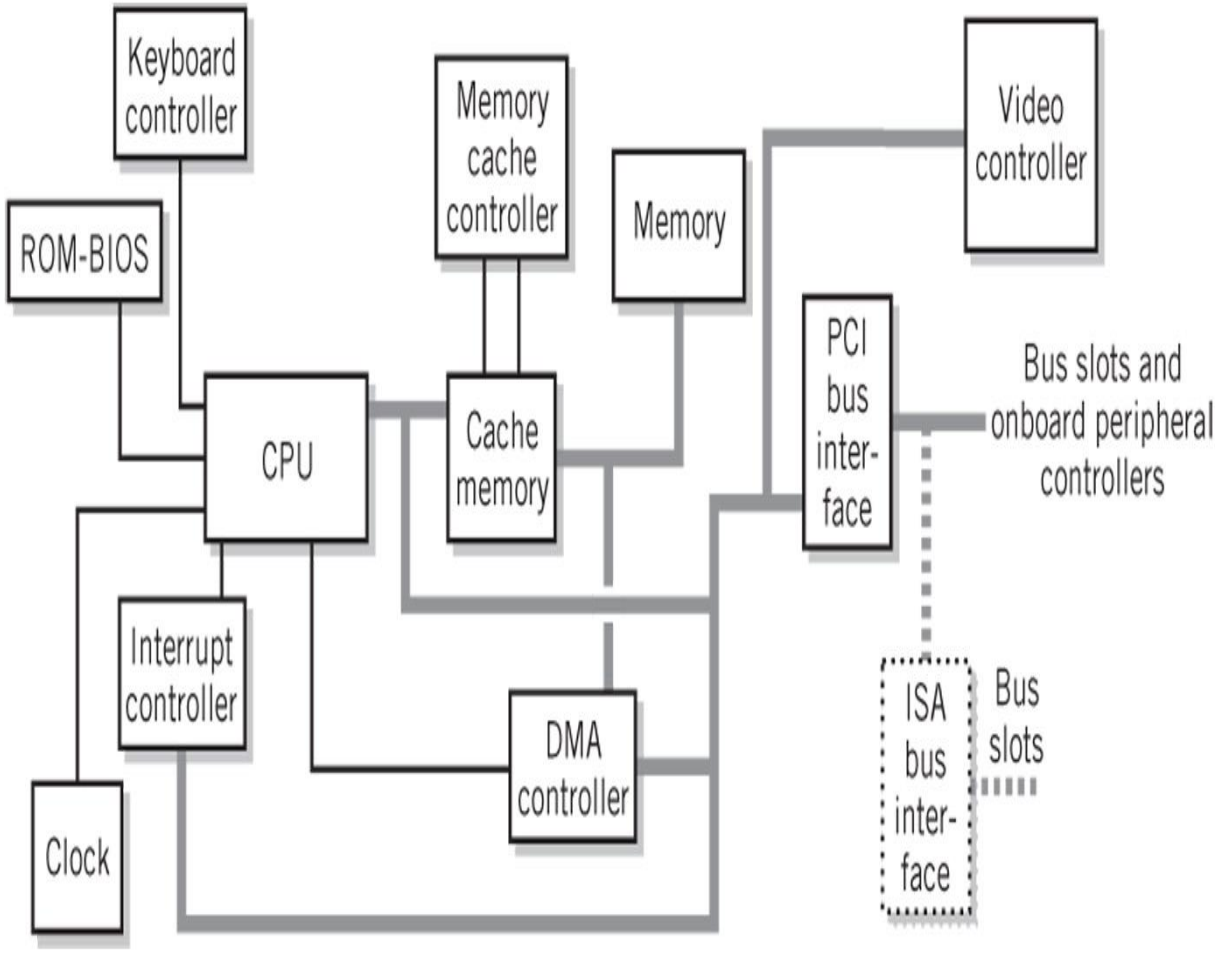





# 1.1 컴퓨터 기본 구조

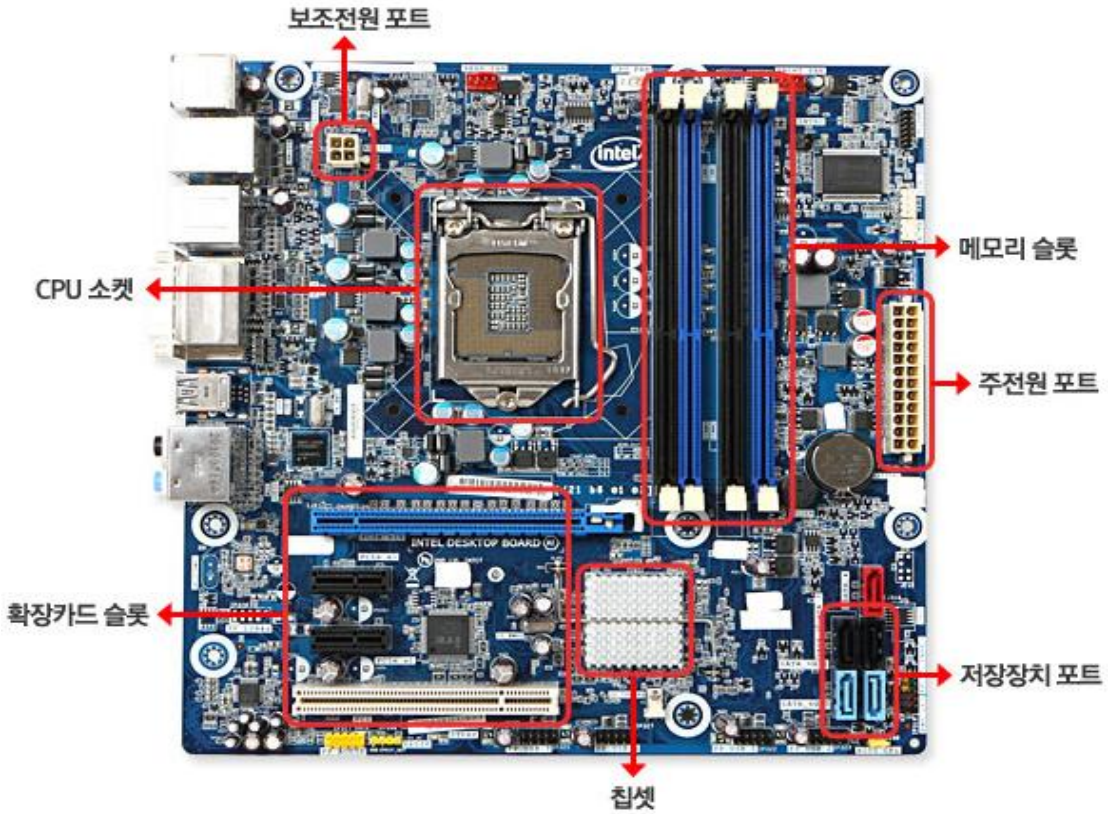


## ■ 참고: 컴퓨터(PC)의 기본적인 구조









# 1.2 정보의 표현과 저장



p18~p21

## ■ 컴퓨터 정보

✓ 2진수 비트들로 표현된 프로그램 코드와 데이터

8086 니모닉스

ARM 니모닉스

8086 데이터북

## ■ 실행 코드

✓ 기계어(machine language = 명령어)로 구성

✓ 기계 코드(machine code)라고도 함

✓ CPU(중앙처리장치)가 이해할 수 있는 유일한 언어이며 2진 비트들로 구성

## ■ 고급 언어(high-level language)

✓ 영문자와 숫자로 구성되어 사람이 이해하기 쉬운 언어

✓ C, C++, JAVA, PYTHON, PASCAL, FORTRAN 등

✓ 컴파일러(compiler) 또는 인터프리터(interpreter)를 이용하여 기계어로 번역되어 실행

## ■ 어셈블리 언어(assembly language)

✓ 고급 언어와 기계어 사이의 중간 언어

➢ **니모닉스**(Mnemocics)를 사용

✓ 어셈블러(assembler)로 번역되며, 기계어와 1:1로 대응

**순수한 의미의 기계어(명령어)는 2진수 형태**

이진수로 표현되는 명령어를 프로그래머가 코딩, 해석하기가 어려워 어셈블리 언어에서는 니모닉스를 사용하는데 이를 그냥 기계어(명령어) 라고 부르기도 한다.

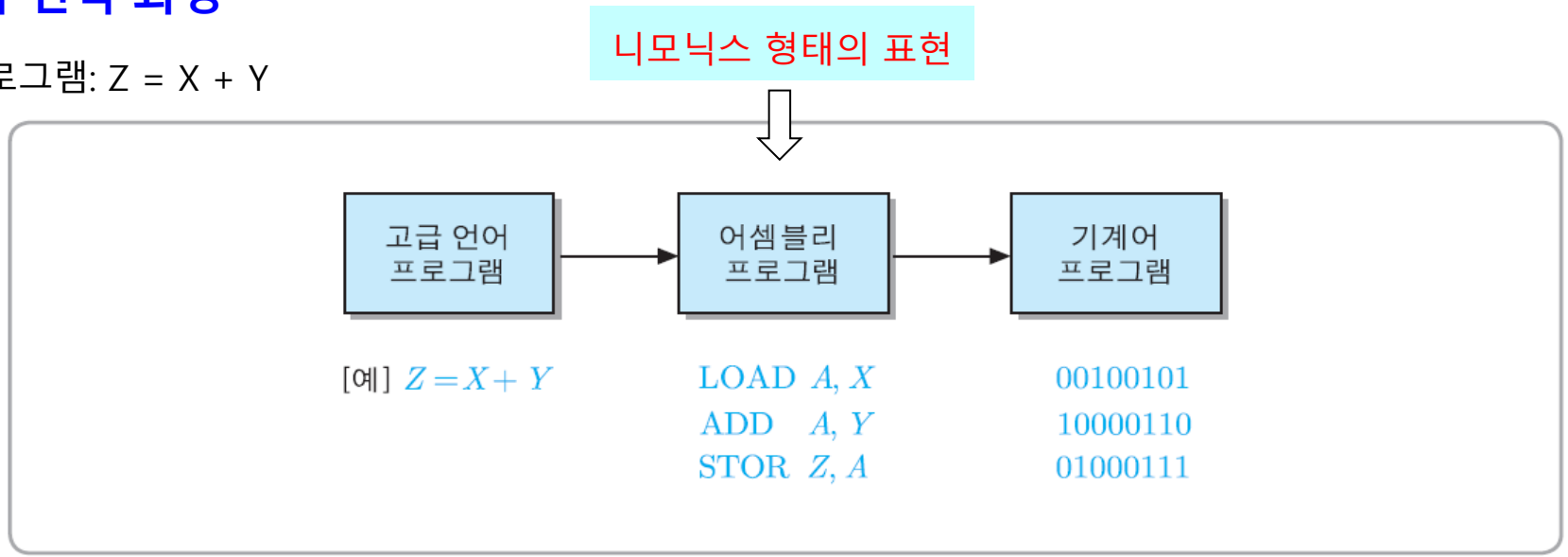
니모닉스 형태로 프로그래밍을 할 수 있는 언어가 어셈블리 언어이다.

# 1.2 정보의 표현과 저장

p18~p21

## ■ 프로그램 언어의 번역 과정

✓ [예] 고급언어 프로그램: Z = X + Y




① LOAD A, X

❖ 기억장치 X번지의 내용을 읽어서, 레지스터 A에 적재(load)

② ADD A, Y

❖ 기억장치 Y번지 내용을 읽어서, 레지스터 A에 적재된 값과 더하고, 결과를 다시 A에 적재

③ STOR Z, A

❖ 그 값을 기억장치 Z 번지에 저장(store)

# 1.2 정보의 표현과 저장

p18~p21

## ■ 프로그램 언어 번역 소프트웨어

✓ 컴파일러(compiler) & 인터프리터(Interpreter)

➢ 고급언어 프로그램을 기계어 프로그램으로 번역하는 소프트웨어

✓ **어셈블러(assembler)**

➢ 어셈블리 프로그램을 **기계어** 프로그램으로 번역하는 소프트웨어

➢ **니모닉스**(mnemonics)를 사용

❖ 이진수인 기계어를 어셈블리 명령어가 사용하는 알파벳 기호 형태로 표현하는 방식 → CPU 제조업체에서 형식을 지정

❖ 'LOAD', 'ADD', 'STOR' 등

❖ 2진수인 기계어를 사용하여 직접 코딩, 디버깅하면 무슨 내용인지를 전혀 모르기 때문에 사용

```
muli $2, $5,4
add  $2, $4,$2
lw   $15, 0($2)
lw   $16, 4($2)
sw   $16, 0($2)
sw   $15, 4($2)
jr   $31
```

1 : 1

```
00000000101000010000000000011000
00000000000011000000110000100001
10001100011000100000000000000000
10001100111100100000000000000100
10101100111100100000000000000000
10101100011000100000000000000100
00000011111000000000000000001000
```

어셈블리 언어(니모닉스)　　기계어 (2진수)

✓ 어셈블리 언어의 사용 목적

1) 속도와 크기가 매우 중요한 프로그램
2) 속도가 매우 중요한 프로그램의 일부분
3) 오래된 컴퓨터 혹은 특수 목적의 컴퓨터

# 1.2 정보의 표현과 저장

## ■ 참고: 고급언어에서 기계어로 변환/실행

✓ 고급언어: 컴파일러 S/W → 목적 파일(object file) → 링커 S/W → 기계어 → 로더 S/W → 실행

✓ 어셈블리 언어: 어셈블러 S/W → 목적 파일 → 링커 S/W → 기계어 → 로더 S/W → 실행

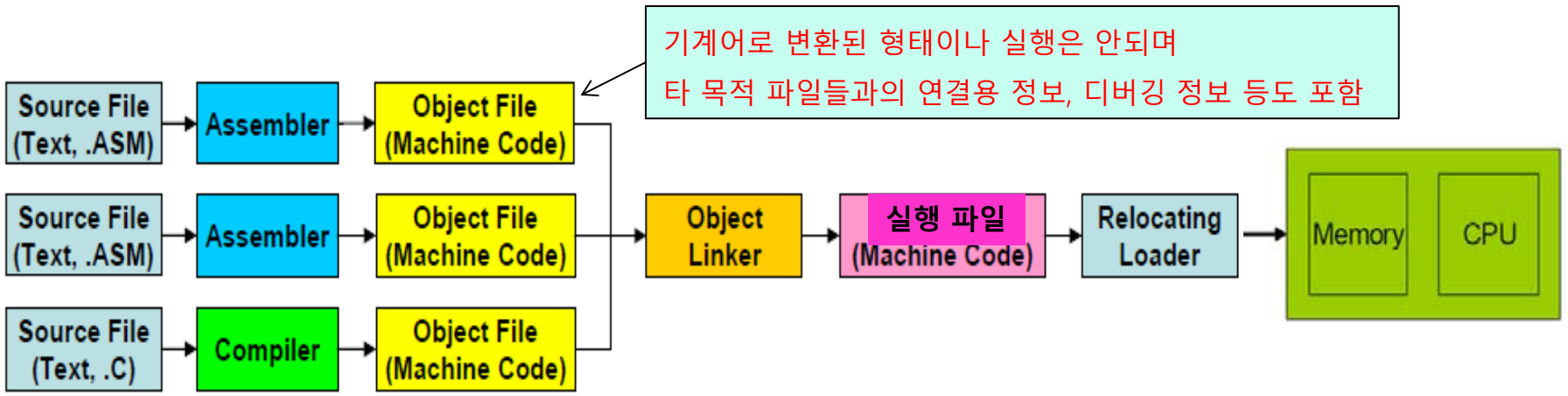
**고급언어 프로그램**

```
1. int n, sum;
2. n = 1;
3. sum = 0;
4. while (n <= 100) {
5.     sum = sum + n;
6.     n = n + 1;
7. }
```

```
    address   machine code     assembly language
    ------------------------------------------------
1.  0041138E  c745f800000001 mov   dword ptr [n],1
2.  00411395  c745ec00000000 mov   dword ptr [sum],0
3.  0041139C  837df864       cmp   dword ptr [n],64h
4.  004113A0  7f14           jg    wmain+46h (4113B6h)
5.  004113A2  8b45ec         mov   eax,dword ptr [sum]
6.  004113A5  0345f8         add   eax,dword ptr [n]
7.  004113A8  8945ec         mov   dword ptr [sum],eax
8.  004113AB  8b45f8         mov   eax,dword ptr [n]
9.  004113AE  83c001         add   eax,1
10. 004113B1  8945f8         mov   dword ptr [n],eax
11. 004113B4  ebe6           jmp   wmain+2Ch (41139Ch)
```

**기계어**

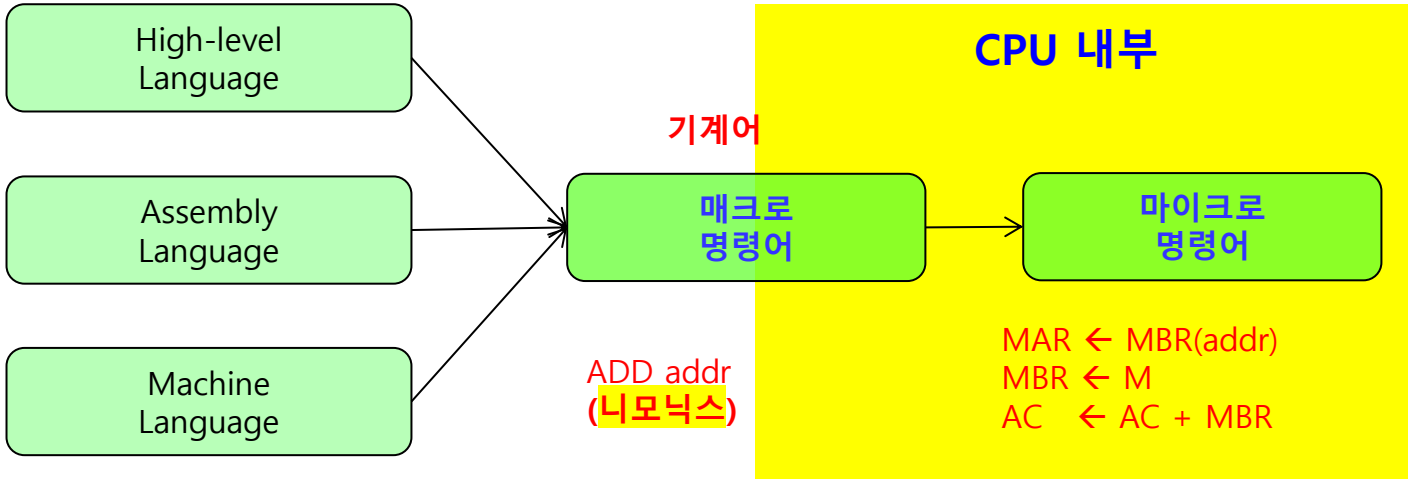
```
00000000101000010000000000011000
00000000000110000011000000100001
10001100011000100000000000000000
10001100111100100000000000000100
10101100111100100000000000000000
10101100011000100000000000000100
00000011111000000000000000001000
```

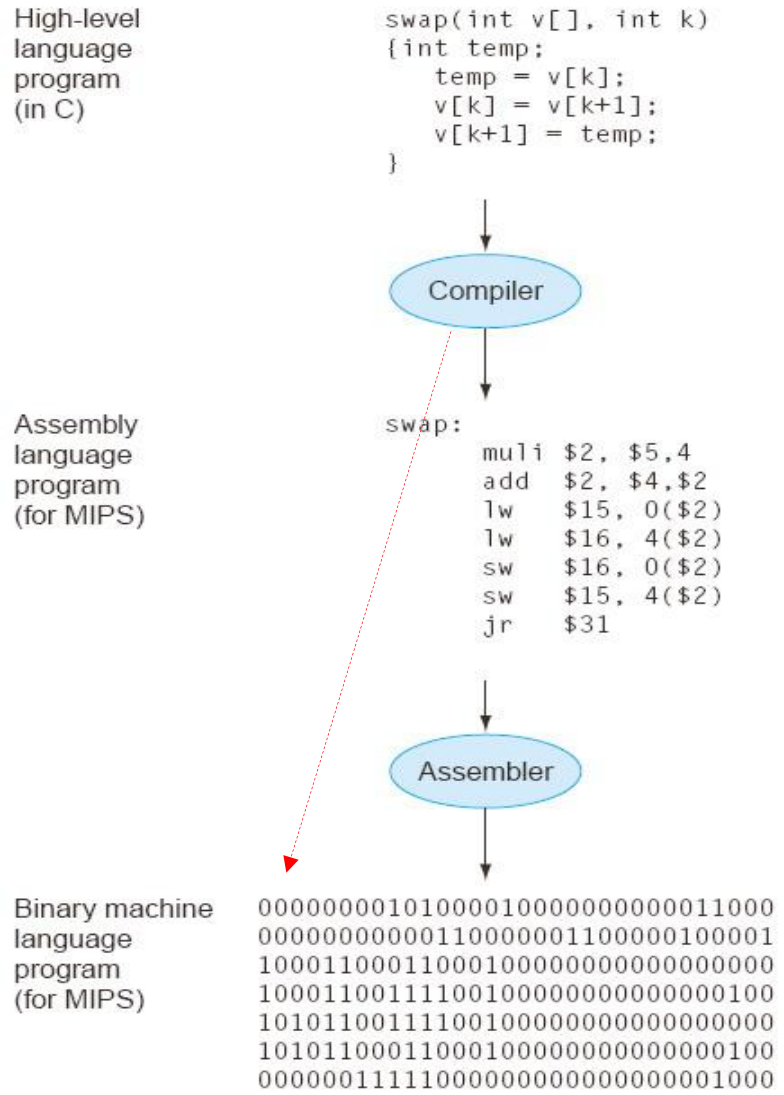


# 1.2 정보의 표현과 저장

## ■ 참고: 명령어의 종류

1) 매크로 명령어 (Macro Instruction)
   - ➢ 우리가 일반적으로 말하는 기계어
2) 마이크로 명령어 (Micro Instruction)
   - ➢ 매크로 명령어를 처리하기 위한 CPU 내부의 세부 명령어
   - ➢ CPU 내부 ROM에 저장되며 프로그래머에게는 보이지 않음
   - ➢ **마이크로 프로그램 = 마이크로 명령어들의 집합**



# 1.2 정보의 표현과 저장



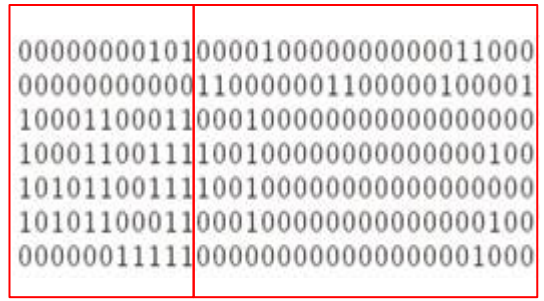
p18~p21

## ■ 명령어(기계어)의 형식

상세 내용은 2장에서 학습

- ✓ 2진수인 명령어(기계어)는 연산코드(연산자) 필드와 오퍼랜드 필드로 구성
- ✓ **CPU 설계시에 결정되며 CPU 마다 연산코드/오퍼랜드 필드의 크기가 다름**

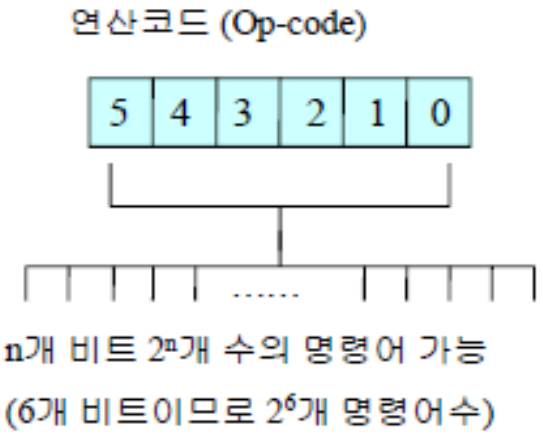



- ✓ **연산 코드(Op-code: Operation code)**
  - ➢ CPU가 수행할 연산(명령)의 종류를 지정해 주는 비트들
  - ➢ 비트 수 = '3' 이면, 지정될 수 있는 연산의 최대 수: $2^3$ = 8개
- ✓ **오퍼랜드(Operand)**
  - ➢ 연산 코드가 실행되는데 필요한 숫자, 주소 등의 정보를 포함하는 필드
  - ➢ 연산코드에 따라 오퍼랜드의 종류가 결정된다
  - ➢ 비트 수 = '5' 라면, 주소지정(addressing) 할 수 있는 기억 장소의 최대 수: $2^5$ = 32 개

# 1.2 정보의 표현과 저장

## ■ 연산코드(op-code)



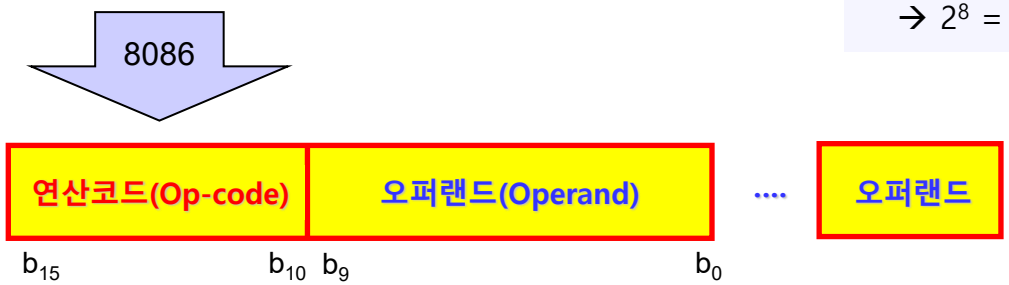

8086 기계어

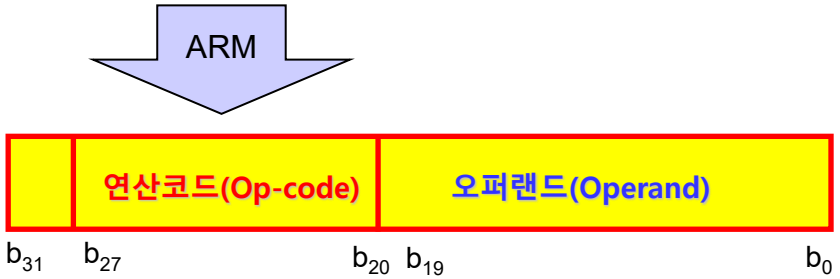
인텔 8086 CPU: 가변길이 명령어 (CISC의 특징)

- 6 비트의 연산코드를 가진다
  → $2^6$ = 64 가지의 연산을 정의 가능
- 명령어 길이는 1~6(8~48비트) Byte로 명령어마다 다름
  → 연산코드에 따라 오퍼랜드의 길이가 달라짐 → 다양한 문제 발생

ARM 프로세서: 고정길이 명령어 (RISC의 특징)

- 명령어의 길이는 항상 32비트(4 Byte=1워드)
- 8 비트의 연산코드
  → $2^8$ = 256 가지의 연산을 정의 가능



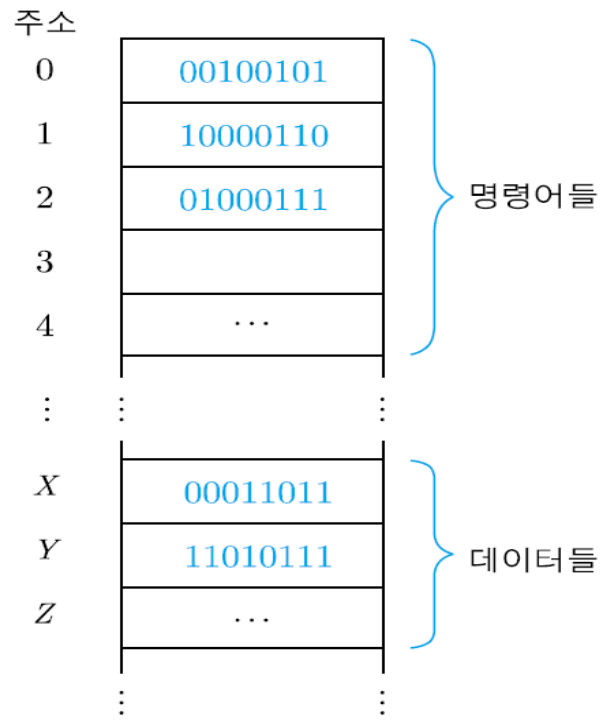
# 1.2 정보의 표현과 저장

p18~p21

## ■ 프로그램 코드와 데이터의 기억장치 저장

교재에서는 '단어'라는 용어를 사용했으나
일반적으로는 '워드(word)'라는 용어를 많이 사용

- ✓ 프로그램 코드(명령어)는 지정된 기억 장소에 순차적으로 저장
- ✓ CPU는 기억장치에 저장된 명령어를 차례대로 하나씩 읽어서 실행
- ✓ **워드(word)는 CPU 내부에서 한 번에 처리될 수 있는 비트들의 수**
  - ➢ CPU 마다 워드의 크기가 다름
- ✓ 명령어(코드)와 데이터는 워드 단위로 기억장치에 저장되며 주소는 주로 바이트 단위로 접근
  - ➢ 워드는 CPU가 한번에 처리 가능한 기본 단위로서, CPU 구조에 따라 워드의 길이가 다르다
    - ❖ 64(32/16/8)비트 CPU의 한 word는 64(32/16/8) 비트
  - ➢ CPU의 메모리 입출력은 보통 워드 단위로 처리하며, 하나의 메모리 주소는 바이트(Byte) 단위로 지정
    - ❖ 32비트(4바이트)의 워드를 저장하기 위해서는 4개의 주소 영역이 필요




일부 MCU는 MCU 내부에 저장되는 프로그램 코드들을 바이트 단위가 아닌 워드 단위로 주소지정을 하는 경우도 있음

# 1.2 정보의 표현과 저장

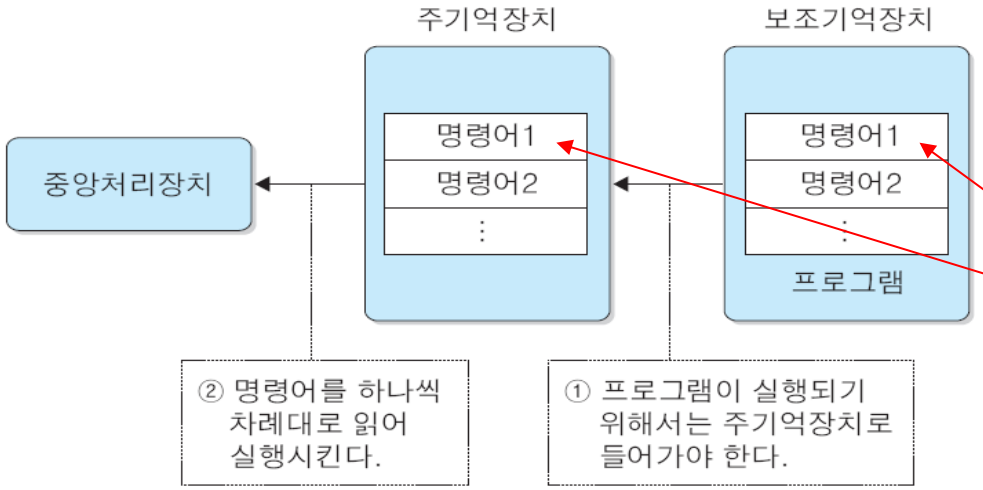
## ■ 컴퓨터 프로그램의 실행

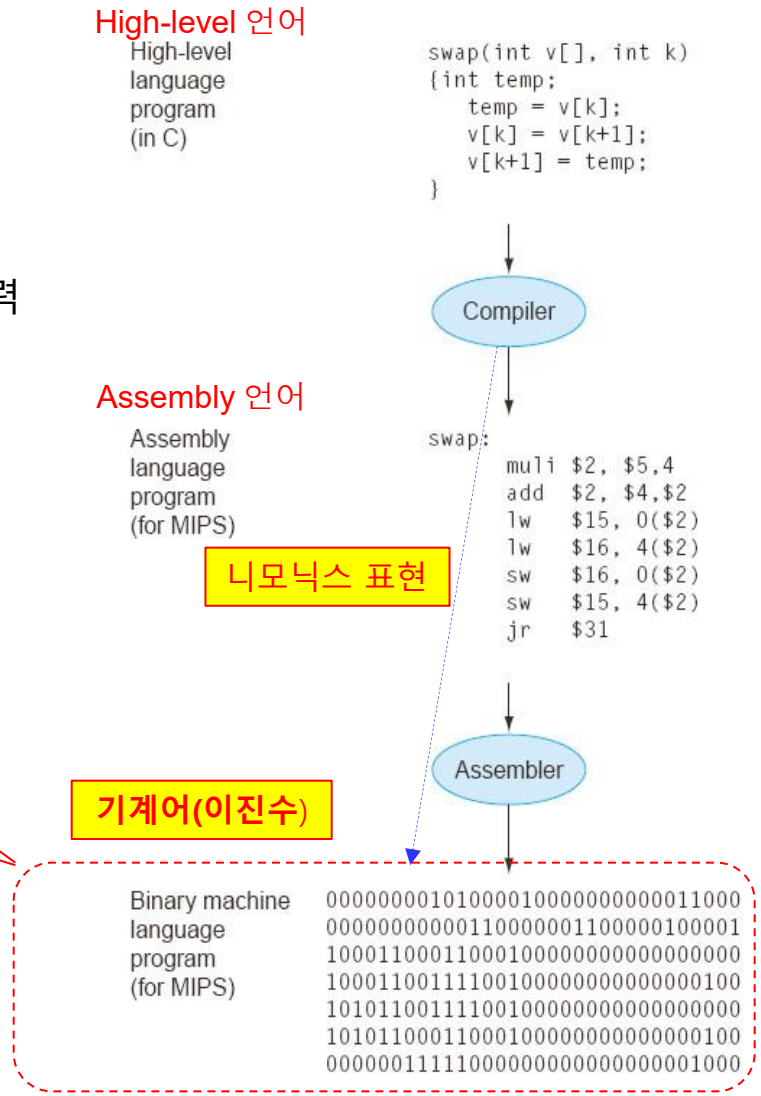
✓ **Stored Program** (폰 노이만 구조)

➢ 실행될 프로그램의 **기계어(명령어)들이** 미리 **순서대로 주기억장치에 저장**되어 있는 구조

➢ 초기의 컴퓨터는 하나의 명령어를 실행할 때마다 스위치를 개별적으로 입력

✓ 프로그램 실행 절차



# 1.3 시스템의 구성

p21~p24

## ■ 시스템 버스(System Bus)

✓ CPU 외부의 모든 부품(기억장치 및 입출력 장치)들은 시스템 버스를 통해 CPU와 연결된다.

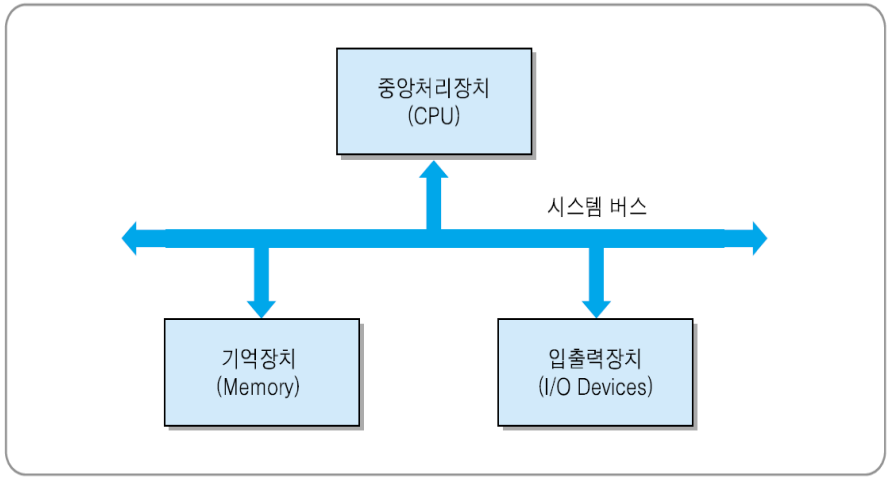
✓ 시스템 버스

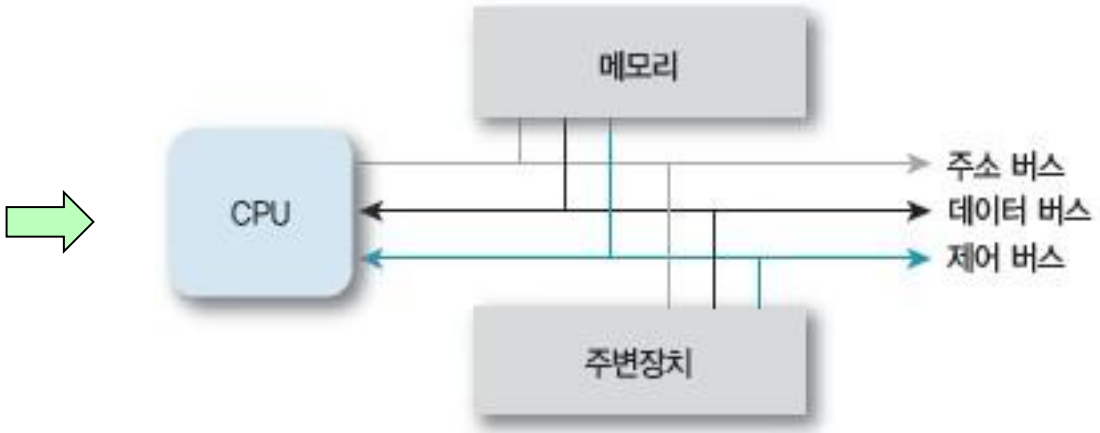
➢ 시스템 내에서 **CPU와 CPU 밖의 다른 요소들 사이에 정보를 교환하는 통로** → **전기 신호선들의 묶음**

➢ 시스템 버스의 기본 구성

1) 주소 버스(address bus)
2) 데이터 버스(data bus)
3) 제어 버스(control bus)



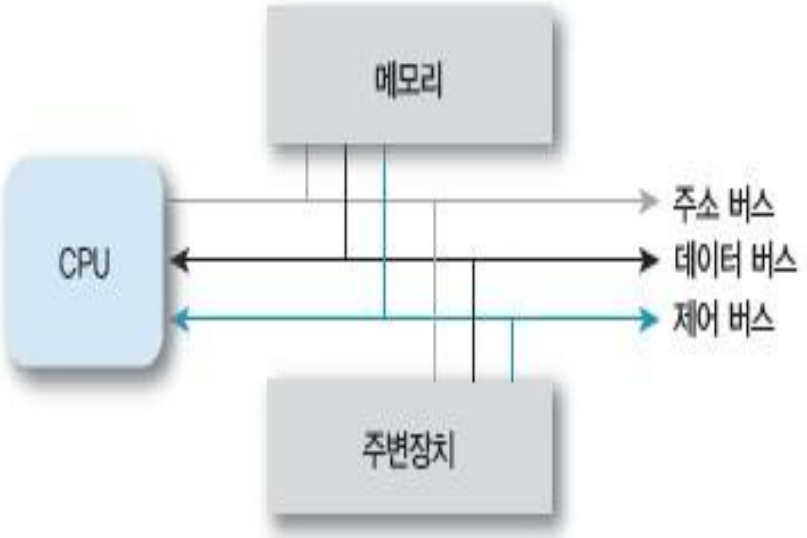
# 1.3 시스템의 구성

p21~p24

## ■ 시스템 버스(System Bus) – 계속

### 1) 주소 버스(address bus)

- CPU로부터 기억장치 혹은 I/O(입출력) 장치들로 보내지는 **주소 신호선**들의 집합
- 단방향성(unidirectional) → CPU에서만 주소를 생성한다(DMA 제외)
- 주소선의 수는 CPU와 접속될 수 있는 최대 기억장치 용량을 결정
- [예] 주소 버스의 비트 수 = 16 비트라면
  - 최대 $2^{16}$ = 64K 개의 기억 장소들의 주소지정 가능
  - $A_{15}$ ~ $A_0$의 16개의 주소선으로 읽기/쓰기를 원하는 주소를 CPU에서 발생




### 2) 데이터 버스(data bus)

- CPU가 기억장치 혹은 I/O 장치와의 사이에 **데이터를 전송하기 위한 신호선들**의 집합
- 양방향성(bidirectional) → 양쪽 방향으로 이동 가능
- 데이터 선의 수는 CPU가 한 번에 전송할 수 있는 비트 수를 결정
- [예] 데이터 버스 폭 = 32비트라면
  - CPU와 기억장치 간의 데이터 전송은 한 번에 32비트씩 가능
  - $D_{31}$ ~ $D_0$의 32개의 데이터 선으로 데이터가 한번에 이동

# 1.3 시스템의 구성

p21~p24

## ■ 시스템 버스(System Bus) – 계속

### 3) 제어 버스(control bus)

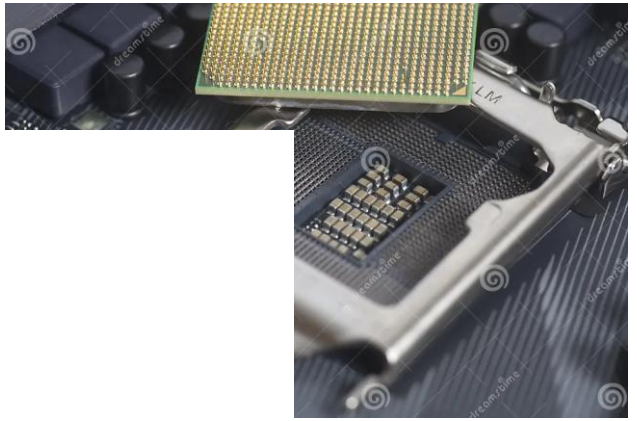
- CPU가 시스템 내의 각종 요소들의 동작을 **제어하기 위한 다양한 신호선들**의 집합
- 다양한 기능의 개별 신호선들로 구성
  - CPU 핀들 중에서 주소버스, 데이터 버스, 전원, 그라운드 및 클럭 등의 특수 용도 핀들을 제외한 나머지 신호선들의 묶음
- 신호선의 방향성은 신호에 따라 다름 (일반적으로 단방향이 많음)
  - 기억장치 읽기/쓰기(Memory Read/Write) 신호, I/O 읽기/쓰기(I/O Read/Write) 신호
  - 인터럽트(Interrupt) 신호, 버스 제어(Bus Control) 신호

< 참고: **근래의 PC용 CPU 구조에서는 공통의 시스템 버스 구조를 사용하지 않고 개별 인터페이스 구조를 사용.** → 7장 상세 학습 >
과거의 인텔 CPU에서는 CPU, 메모리, I/O 장치 등 주요 부품들이 공통 시스템 버스를 통해 연결되었고 이때의 시스템 버스는 데이터 버스, 주소 버스, 제어 버스로 구성되는 병렬 버스 구조였다. 그러나 근래의 Intel CPU에서는 이러한 전통적인 시스템 버스의 역할이 크게 축소되었다. 즉 메모리 컨트롤러, PCIe 컨트롤러 등의 기능이 CPU에 통합되고, CPU와 주요 장치 사이에는 DDR 메모리 인터페이스, PCIe, DMI 등의 개별 고속 인터페이스가 사용된다. 따라서 현대 PC는 하나의 공통 시스템 버스를 중심으로 연결되는 구조가 아닌, CPU를 중심으로 여러 전용 인터페이스가 계층적으로 연결되는 구조로 발전하였다. CPU 핀에는 전원·접지뿐만 아니라 DDR5, PCIe, DMI, 디스플레이, 클록 및 각종 제어·관리 신호 등이 포함된다.
우리는 인텔 CPU는 물론 전체 CPU를 대상으로 학습하기 때문에 **개별 인터페이스들도 편의상 그냥 시스템 버스의 일부로 포함시켜 학습함**.
참고로 근래의 PC용 Intel 64비트 CPU의 소켓 이름(LGA1851 등)의 숫자는 CPU와 메인보드 사이의 접점 수를 의미한다.

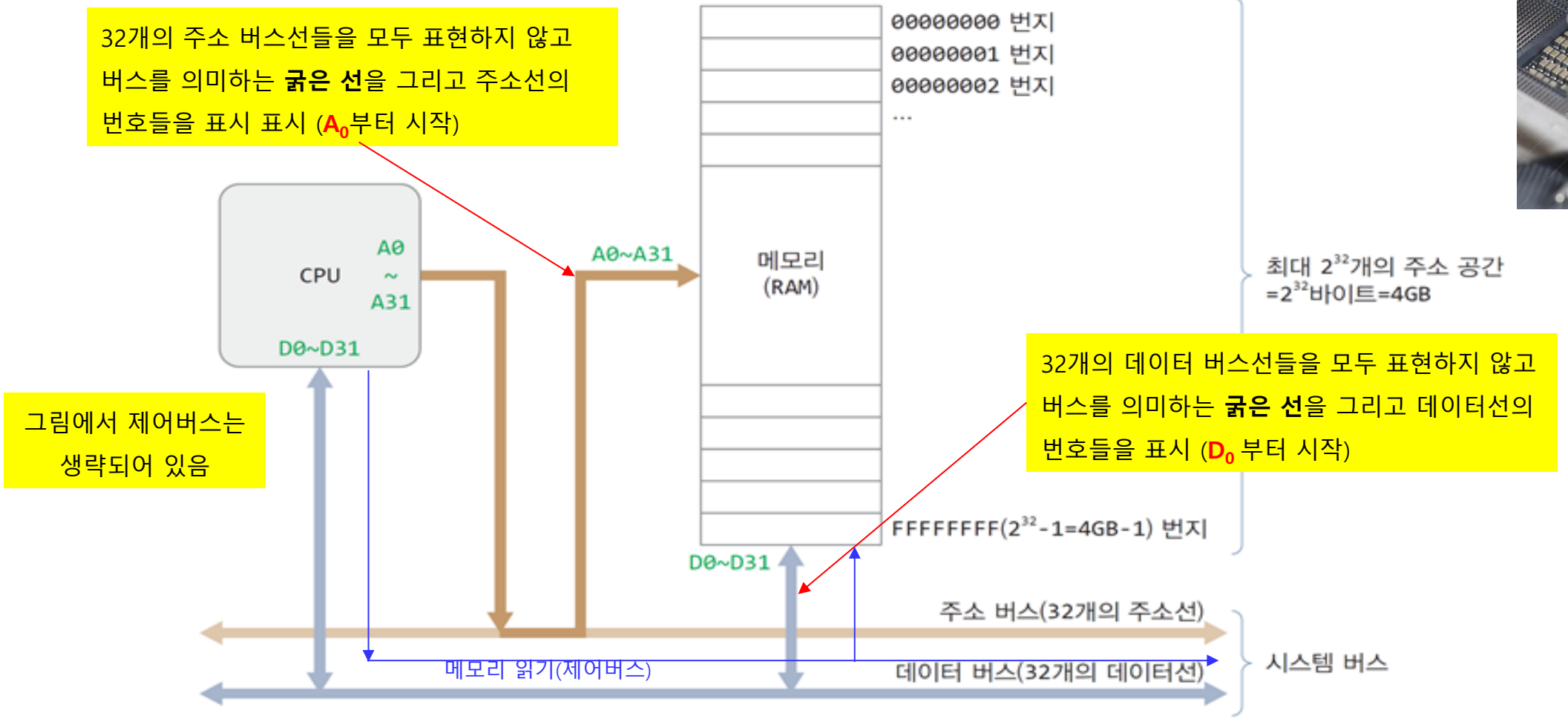
# 1.3 시스템의 구성

## ■ CPU와 주기억장치 간의 접속

✓ 32비트의 주소 버스와 데이터 버스를 갖는 CPU의 예



32개의 주소 버스선들을 모두 표현하지 않고 버스를 의미하는 **굵은 선**을 그리고 주소선의 번호들을 표시 표시 ($A_0$부터 시작)




그림에서 제어버스는 생략되어 있음

32개의 데이터 버스선들을 모두 표현하지 않고 버스를 의미하는 **굵은 선**을 그리고 데이터선의 번호들을 표시 ($D_0$ 부터 시작)

p21~p24

## ■ CPU와 주기억장치 간의 접속 - 계속

✓ 주기억장치 쓰기 동작

➢ CPU가 데이터를 저장할 기억 장소의 주소와 저장할 데이터를 각각 주소 버스와 데이터 버스를 통하여 보내는 동시에, (메모리) 쓰기 신호를 활성화

✓ 주기억장치 쓰기 시간(memory write time)

➢ CPU가 주소와 데이터를 보낸 순간부터 저장이 완료될 때까지의 시간




(a) 기억장치 쓰기 동작의 시간 흐름도

p21~p24

## ■ CPU와 주기억장치 간의 접속 - 계속

### ✓ 주기억장치 읽기 동작

- ➢ CPU가 기억장치 주소를 주소 버스를 통하여 보내는 동시에, (메모리) 읽기 신호를 활성화
- ➢ 일정 지연 시간이 경과한 후에 기억장치로부터 읽혀진 데이터가 데이터 버스 상에 실리고, CPU는 그 데이터를 버스 인터페이스 회로를 통하여 내부로 읽어음

### ✓ 주기억장치 읽기 시간(memory read time)

- ➢ 주소를 발생한 시간부터 기억장치의 데이터가 CPU에 도착할 때까지의 시간




(b) 기억장치 읽기 동작의 시간 흐름도

# 1.3 시스템의 구성



p24~p27

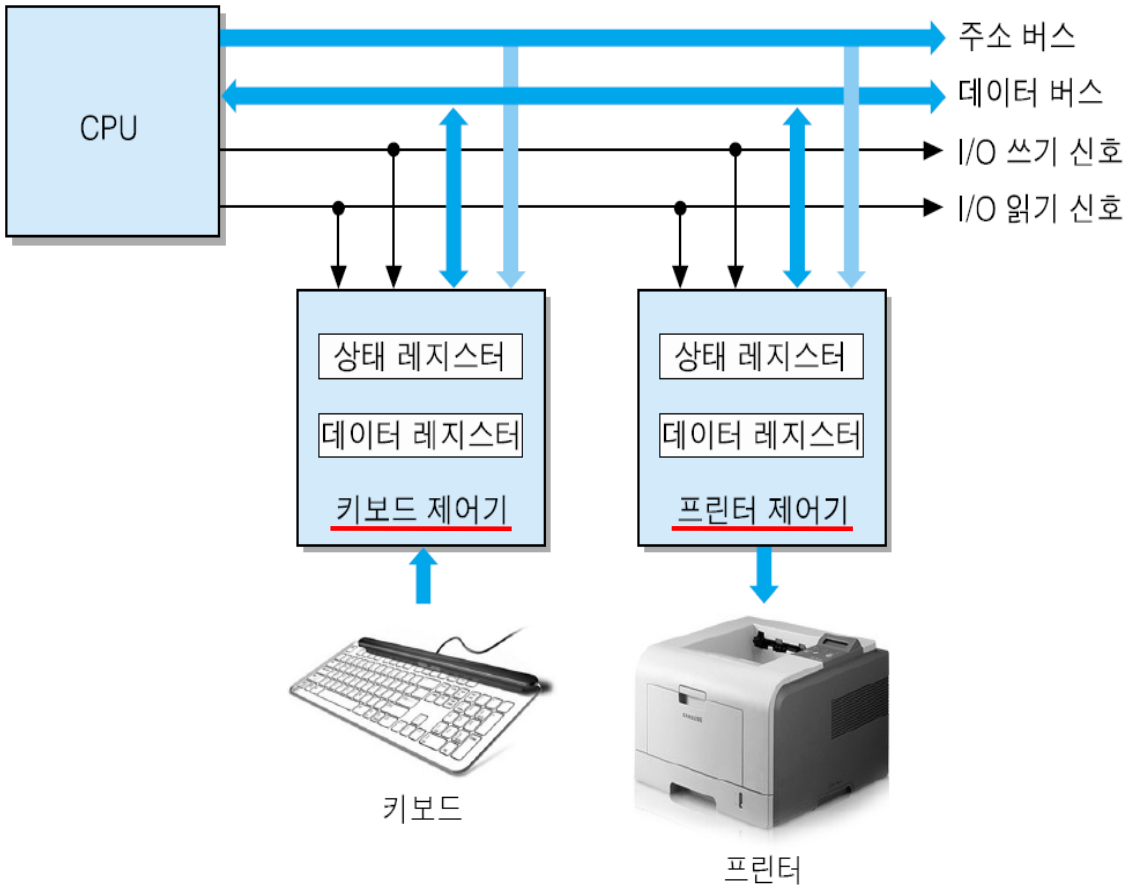
## ■ CPU와 I/O(입출력) 장치의 접속

✓ 필요한 버스 및 제어 신호

1) 주소 버스
2) 데이터 버스
3) 제어 신호 (제어 버스의 일부분)
   - ❖ I/O 읽기 신호, I/O 쓰기 신호

✓ 접속 경로

- ➢ CPU ↔ 시스템 버스 ↔ **I/O 장치 제어기** ↔ I/O 장치
- ➢ CPU에서 다양한 입출력 장치의 내부까지 직접 시스템 버스를 통해 접근하는 것은 불가능 하기 때문에 장치별로 I/O 장치 제어기를 사용
  - ❖ 종류가 많고 각각의 동작 방식들도 다르기 때문




< I/O 장치 접속 사례 : CPU – 키보드 & 프린터 >

# 1.3 시스템의 구성

p24~p27

## ■ I/O 장치 제어기(I/O device controller)

✓ 기능

➢ CPU로부터 I/O 명령을 받아서 해당 I/O 장치를 제어하고, 데이터를 이동함으로써 명령을 수행하는 전자회로 장치

❖ 목적에 따라 동작방식, 인터페이스가 다양한 입출력 장치들을 **표준화**하여 CPU가 시스템 버스를 통해 **통일된 방식으로 접근**할 수 있게 하는 것이 장치 제어기의 목적

➢ 예

❖ 키보드 제어기, 프린터 제어기 등

✓ CPU와 각 I/O 장치 제어기 내부의 레지스터들이 시스템 버스를 통해 연결

➢ 대부분 다음 2 가지의 레지스터를 기본으로 사용

➢ 상태 레지스터(Status Register)

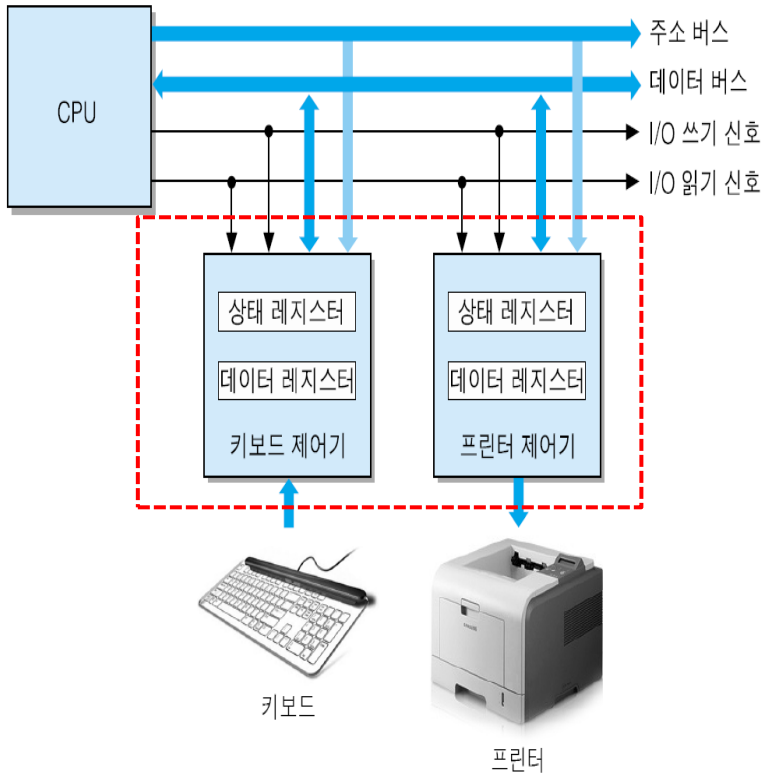
❖ I/O 장치의 현재 상태를 나타내는 비트들을 저장한 레지스터

❖ 레지스터의 주요 내용

» 준비 상태(RDY) 비트, 데이터 전송확인(ACK) 비트, 등

➢ 데이터 레지스터(Data Register)

❖ CPU와 I/O 장치 간에 이동되는 명령어/데이터를 일시적으로 저장하는 레지스터



# 1.3 시스템의 구성

p24~p27

## ■ 키보드의 데이터 입력 과정

### ✓ 키보드 제어기

① 키보드의 어떤 한 키(key)를 누르면,

② 그 키에 대응되는 ASCII 코드가 키보드 제어기의 데이터 레지스터에 저장되고,

③ 동시에 상태 레지스터의 In_RDY 비트가 1로 세트

❖ In_RDY 비트는 데이터 레지스터에 외부로부터 데이터가 적재되었는지를 표시

### ✓ CPU

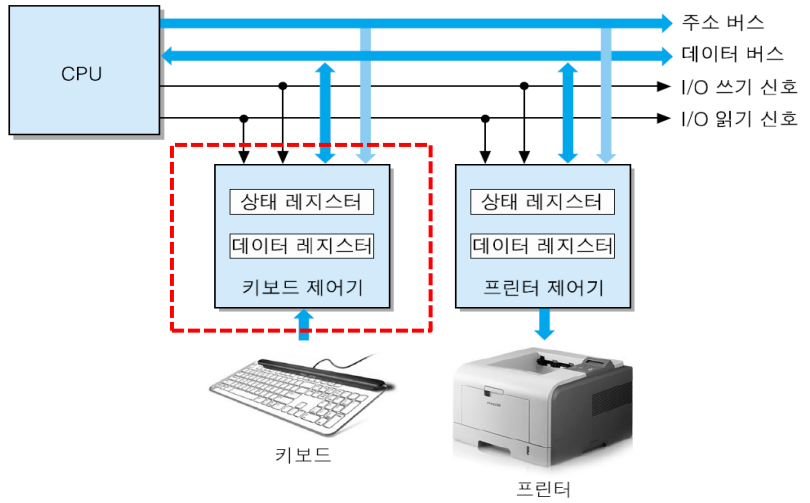
④ 키보드 제어기로부터 상태 레지스터의 내용을 읽어서

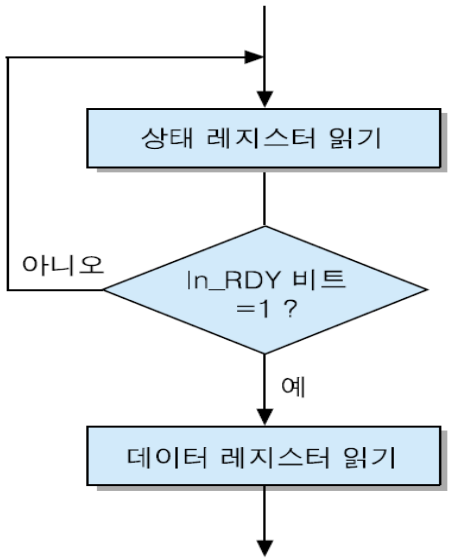
⑤ In_RDY 비트가 세트 되었는지 검사

❖ In_RDY 비트 = 데이터 레지스터에 데이터가 적재

⑥ 만약 세트 되지 않았으면, ④번을 반복하며 대기

⑦ 만약 세트 되었다면, 데이터 레지스터의 내용을 읽음



# 1.3 시스템의 구성

p24~p27

## ■ 프린터의 데이터 출력 과정

✓ CPU

① 프린터 제어기의 상태 레지스터의 내용을 읽고 Out_RDY 비트 검사

❖ Out_RDY 비트 → 프린터가 출력할 준비가 되었는지를 표시

② 만약 세트 되지 않았으면, ①번을 반복하며 대기

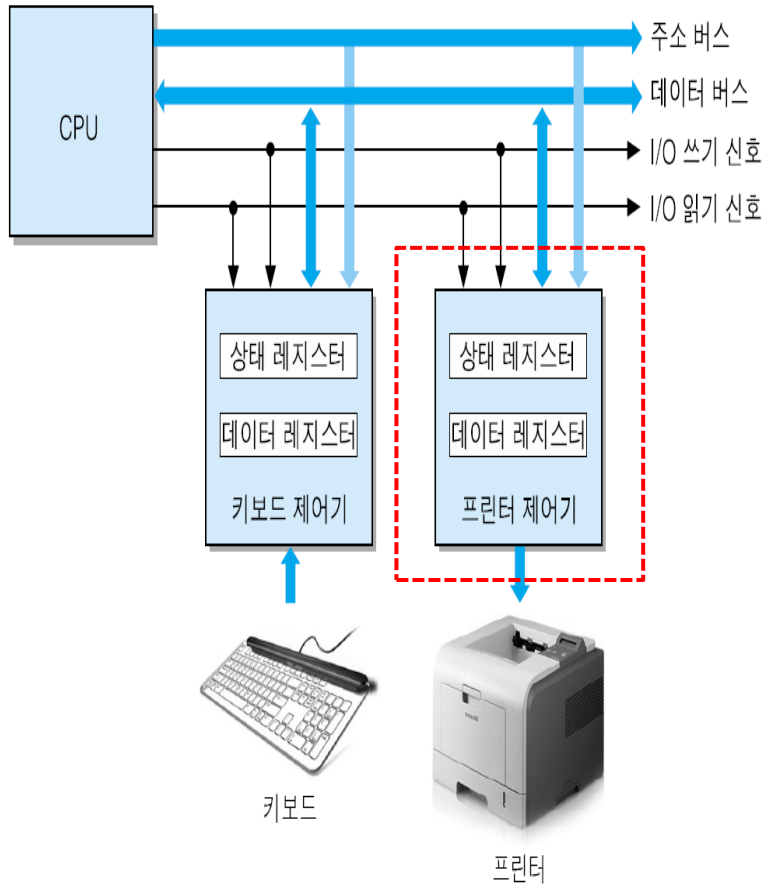
③ 만약 세트 되었다면, 프린트할 데이터를 프린터 제어기의 데이터 레지스터에 저장

❖ 데이터 블록 전체를 제어기 내의 데이터 기억장치에 저장하는 것도 가능 → spooling

✓ 프린터 제어기

① 데이터 레지스터(혹은 데이터 기억장치)의 내용을 프린터로 보내고,

② 프린터의 하드웨어를 제어하면서 인쇄 동작 수행



# 1.3 시스템의 구성

p24~p27

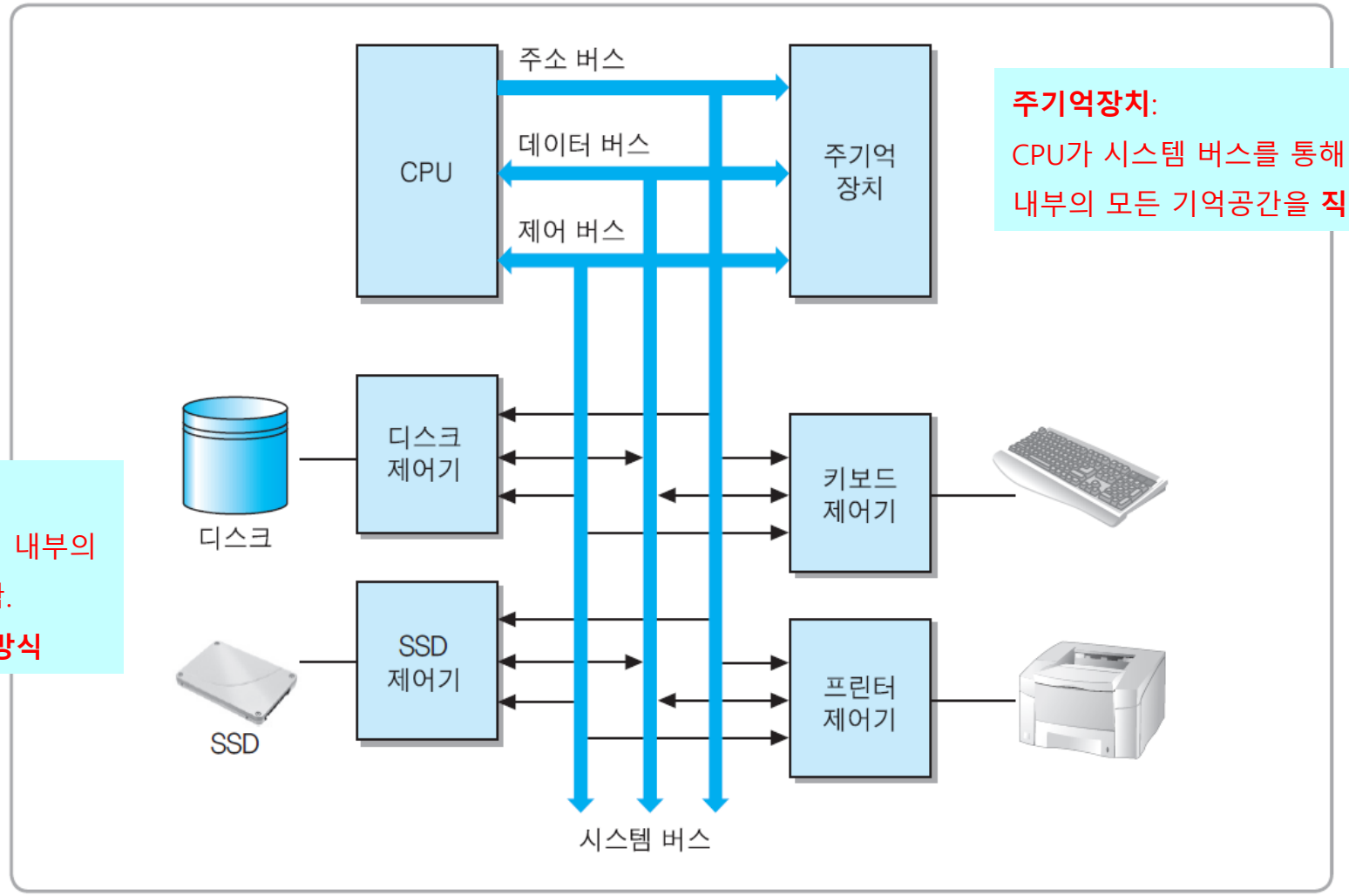
## ■ CPU와 보조저장장치의 접속

- ✓ 보조기억장치라고도 부름
- ✓ 주기억장치와의 가장 큰 차이는 CPU가 보조기억장치 내부의 모든 기억공간을 시스템 버스를 통해 직접 접근하지 못한다는 점
- ✓ 종류
  - ➢ 디스크, 플래시 메모리, SSD, CD-ROM 등
- ✓ 보조저장장치들도 각 장치를 위한 제어기를 통하여 키보드나 프린터 같은 입출력 제어기와 동일한 방법으로 접속
- ✓ 입출력 장치와 보조저장장치의 차이점은 전송 단위 및 장치의 목적
  - ➢ 키보드
    - ❖ 바이트 단위의 입력 데이터 전송
  - ➢ 보조저장장치
    - ❖ 프로그램/데이터를 저장
    - ❖ 블록(512/1024/4096 바이트) 단위로 전송
    - ❖ 제어기 내에는 한 블록 이상을 임시 저장할 수 있는 데이터 기억장치(버퍼)가 필요

# 1.3 시스템의 구성

p27~p29

## ■ 컴퓨터 시스템의 전체 구성




**주기억장치**:
CPU가 시스템 버스를 통해 기억장치 내부의 모든 기억공간을 **직접 접근 가능**

**보조기억장치**:
CPU가 시스템 버스를 통해 기억장치 내부의 모든 기억공간을 직접 접근하지 못함.
장치 제어기를 통한 **간접적인 접근 방식**

# 1.3 시스템의 구성



p27~p29

## ■ 컴퓨터의 기본적인 기능들

1) 프로그램 실행

- CPU가 주기억장치로부터 프로그램 코드(명령어, 기계어)를 읽어서 실행

2) 데이터 저장

- 프로그램 실행 결과로서 얻어진 데이터를 주기억장치에 저장

3) 데이터 이동

- 디스크 혹은 CD-ROM에 저장되어 있는 프로그램과 데이터 블록을 기억장치로 이동

4) 데이터 입력/출력

- 사용자가 키보드를 통하여 보내는 명령이나 데이터를 읽어 들인다.
- 또한 CPU가 처리한 결과값이나 기억장치의 내용을 프린터(혹은 모니터)로 출력

5) 제어

- 프로그램이 순서대로 실행되도록 또는 필요에 따라 실행 순서를 변경하도록 조정하며, 각종 제어 신호들을 발생

# 1.3 시스템의 구성

p27~p29

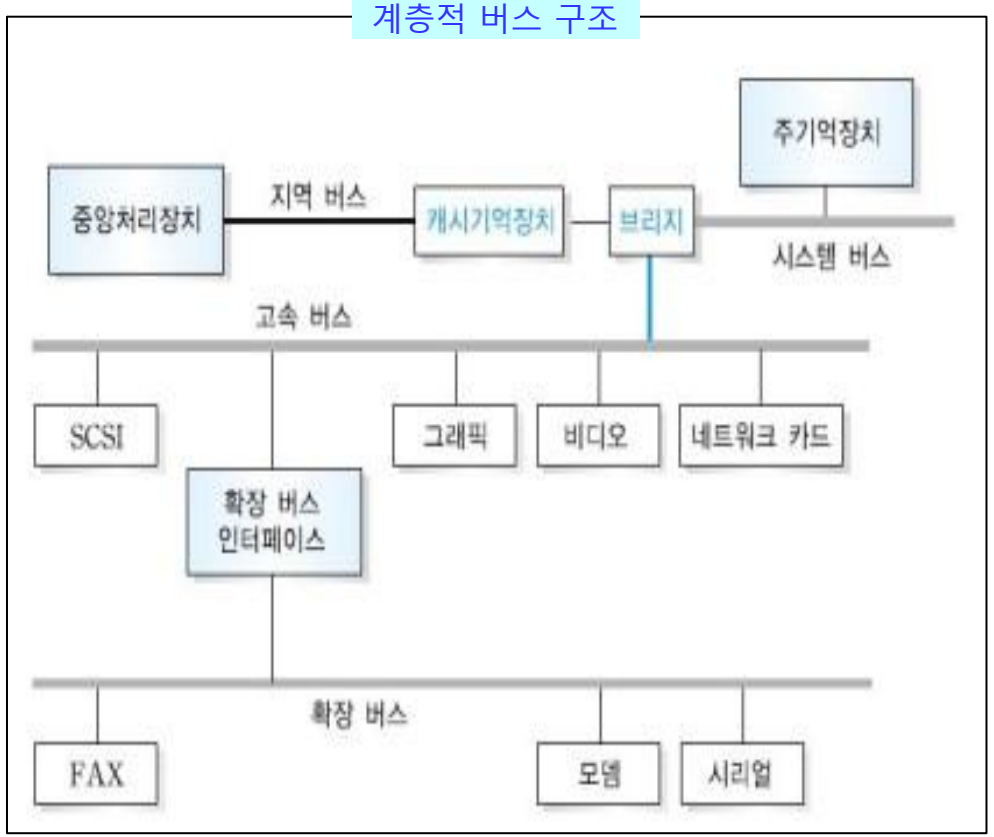


## ■ 확장 버스(Expansion Bus) 구조

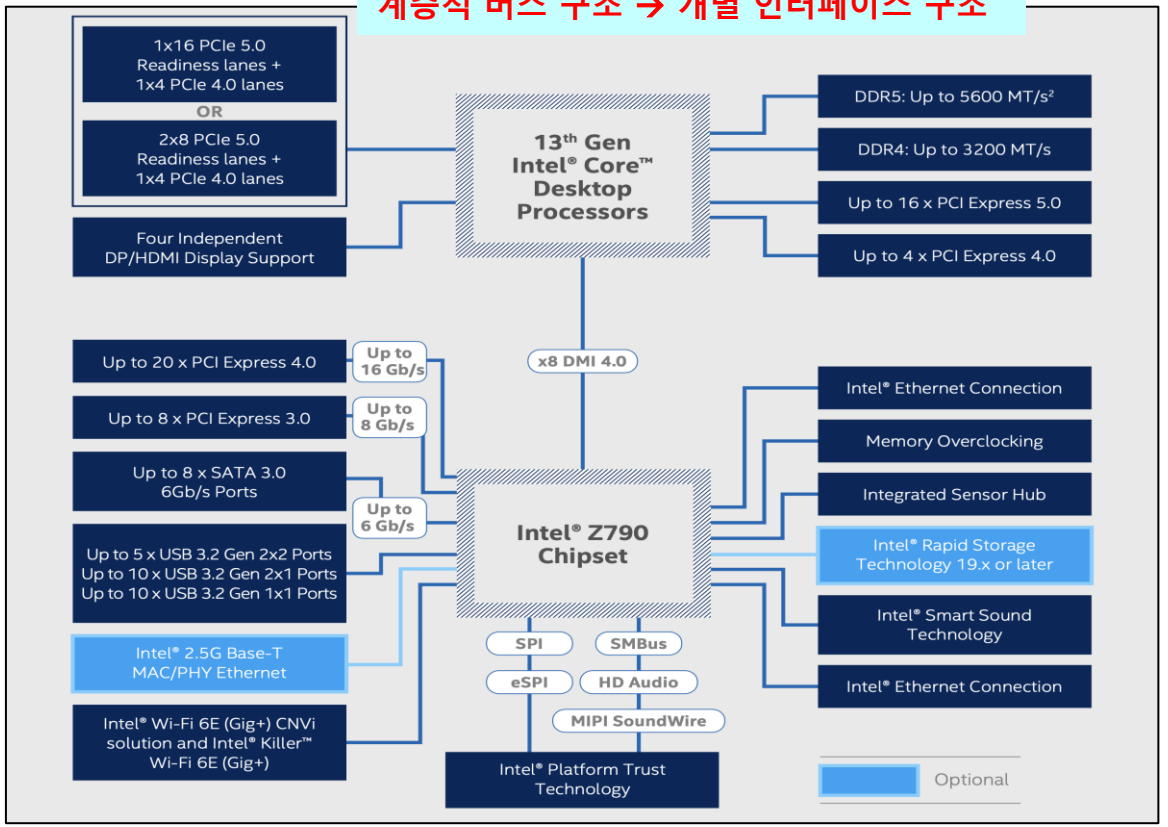
✓ 계층적 버스 구조(Hierarchical bus structure)

➢ 하나의 시스템 버스에 연결되는 주기억장치, 보조기억장치, I/O 장치들은 속도 차이가 많이 나고 또한 버스는 가장 느린 장치를 기준으로 움직여야만 하기 때문에 발생하는 **버스 병목(Bottleneck) 현상**을 해결하는 구조 → 복수개의 계층적 시스템 버스 사용

계층적 버스 구조



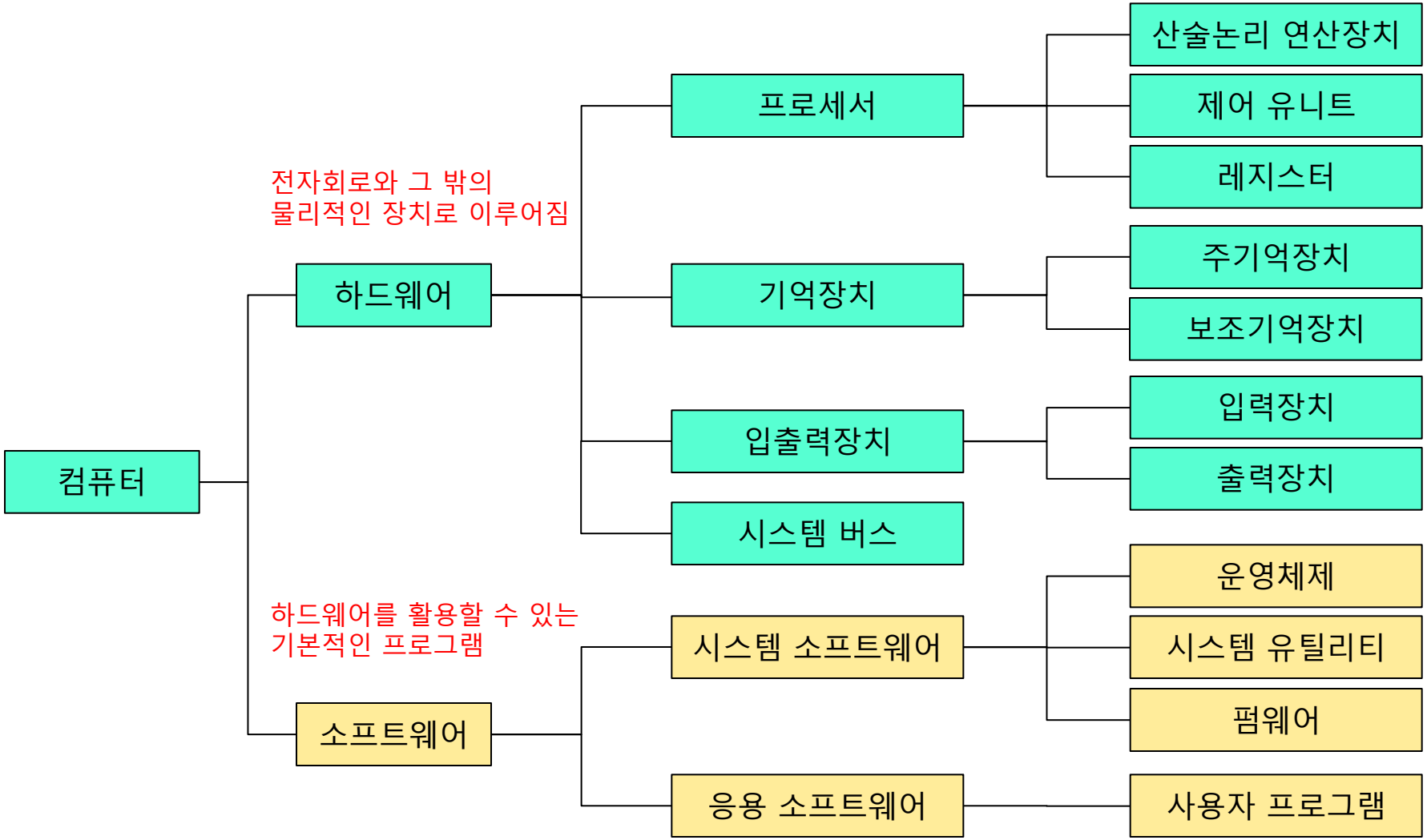

**계층적 버스 구조 → 개별 인터페이스 구조**



# 1.3 시스템의 구성



## ■ 컴퓨터의 기본적인 구성요소 (정리)



# 1.4 컴퓨터 구조의 발전 과정



p29~p34

## ■ 최초의 컴퓨터

- ✓ 1642년, Blaise Pascal(프랑스)
- ✓ 덧셈과 뺄셈을 수행하는 기계적 카운터
- ✓ 다이얼의 위치에 의하여 십진수를 표시하는 6개의 원형 판 세트들로 구성
- ✓ 각 원형판은 일시적으로 숫자를 기억하는 레지스터로 사용



## ■ Difference Engine (차분엔진)

- ✓ 19세기 초, Charles Babbage(영국, 현대 컴퓨터의 할아버지)
- ✓ 표에 있는 수들을 자동적으로 계산하고, 결과를 금속천공기에 프린트
- ✓ 덧셈과 뺄셈만 수행 가능

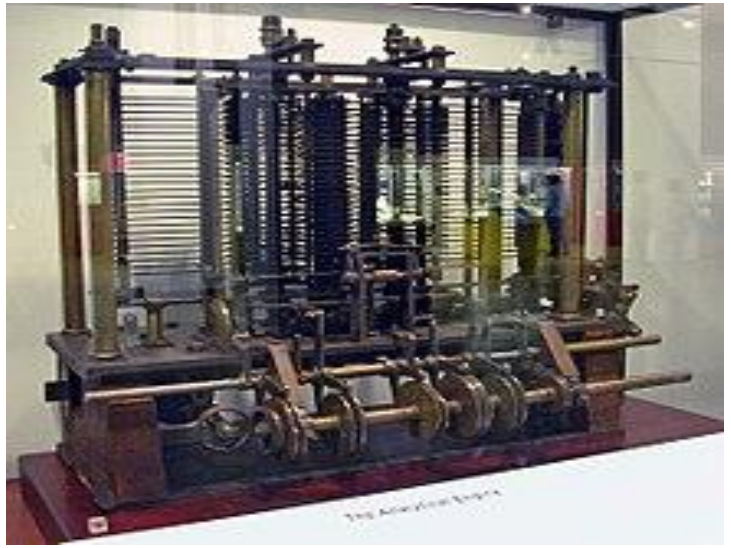


# 1.4 컴퓨터 구조의 발전 과정

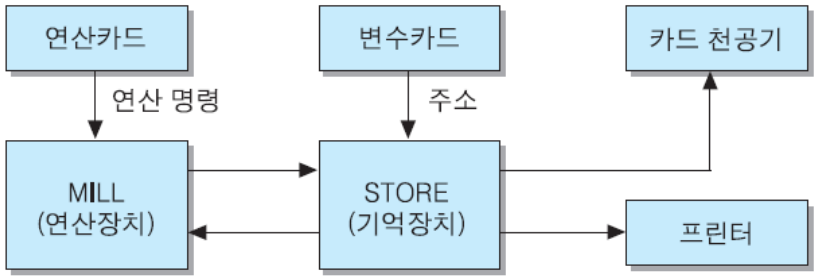


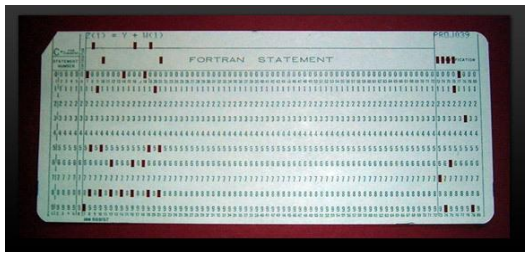
p29~p34

## ■ Analytical Engine (해석엔진)



- ✓ 19세기 초, Charles Babbage(영국)
- ✓ 주요 특징들
  - ➢ 어떤 수학 연산도 자동적으로 수행할 수 있는 일반 목적용 기계식 계산 기계
  - ➢ 프로그래밍 가능: 프로그램 언어 사용
  - ➢ 프로그램의 실행 순서 변경 가능
  - ➢ 수의 부호 검사를 이용한 조건 분기
  - ➢ 제어 카드를 이용한 실행 순서 변경
- ✓ 문제점
  - ➢ 주요 부품들이 기계적 장치라서 속도가 느리고 신뢰도가 낮았음
- ✓ 기본 구조
  - ➢ 산술연산장치: MILL
  - ➢ 기억장치: STORE
  - ➢ 입력장치: 카드 판독기
  - ➢ 출력장치: 카드 천공기, 프린터



< 천공 카드 >

# 1.4 컴퓨터 구조의 발전 과정



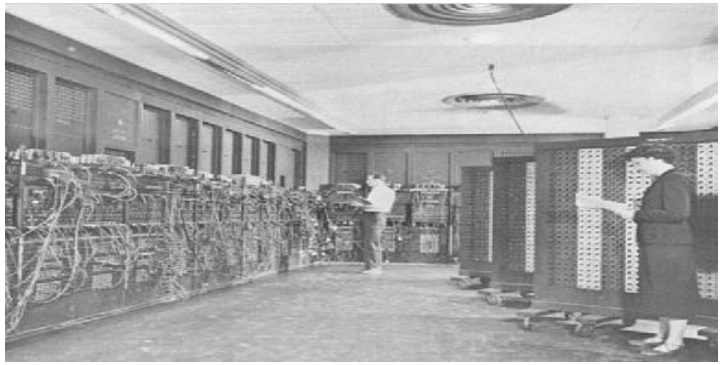
p29~p34

## ■ ENIAC(에니악)

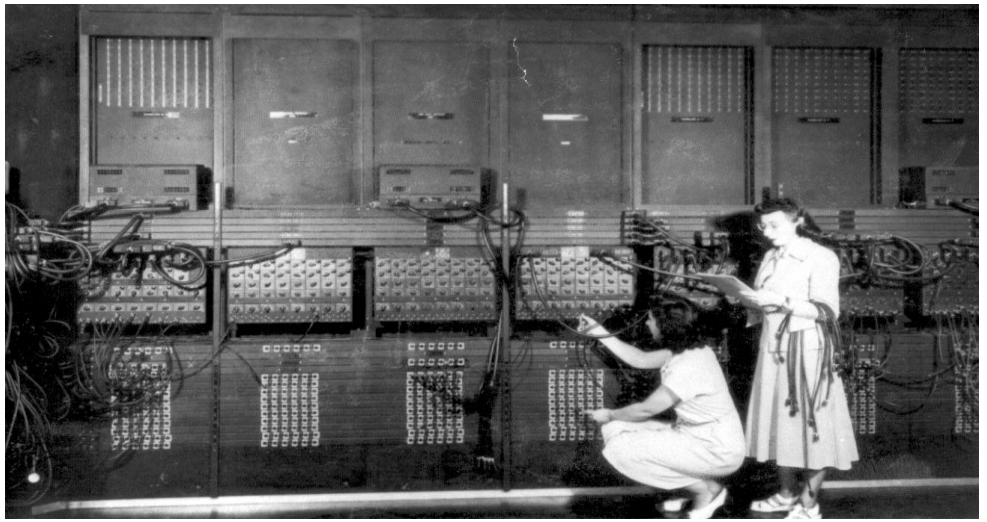
- ✓ 1940년대 초, Von Neumann(폰 노이만)이 개발
- ✓ 펜실베니아 대학에서 개발한 진공관을 사용한 **최초의 전자식 컴퓨터**
- ✓ 문제점: 프로그램의 저장 및 변경 불가능



< 진공관 >



< ENIAC >



< 전선을 연결하여 프로그램을 입력 >

## ■ 폰 노이만의 설계 개념(stored-program 원리) 발표

- ✓ 프로그램과 데이터를 내부에 저장하며 변경이 가능
- ✓ 2진수 체계(binary number system) 사용
- ✓ 1945년 발표 후, EDVAC(Electronic Discrete Variable Computer) 개발에 실제 적용

# 1.4 컴퓨터 구조의 발전 과정



## ■ 폰 노이만 구조(von Neumann Architecture)

### 1) 폰 노이만 구조 – 1

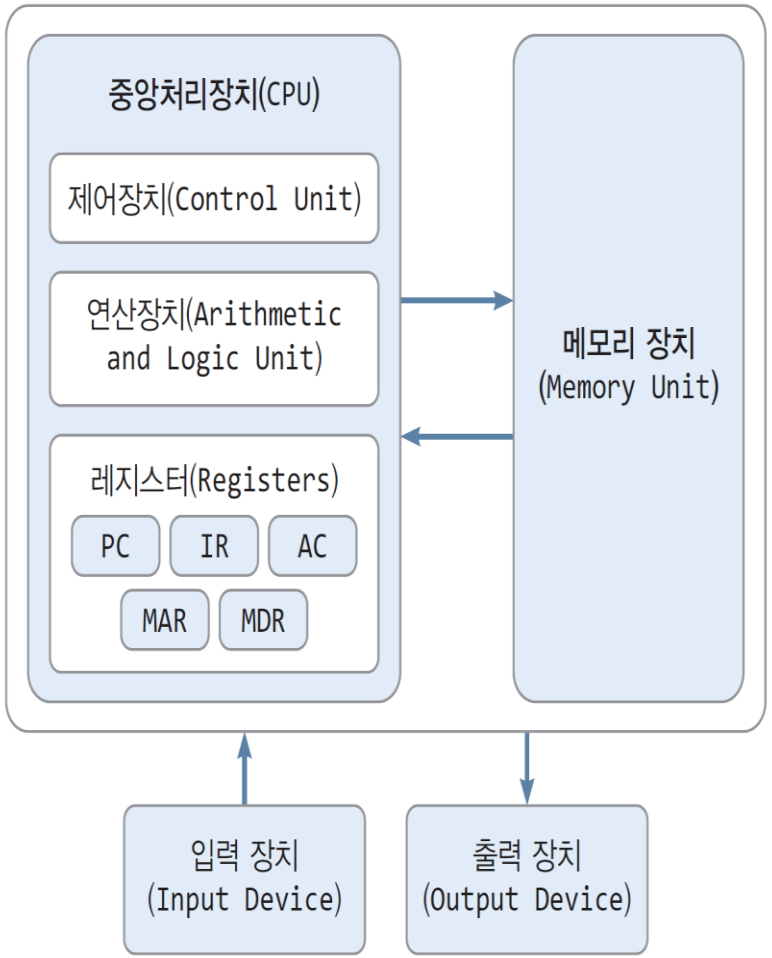
- **Stored Program(내장 프로그램 방식)**이라고도 부름 → **배선작업이 사라짐**
- 2진수로 구성된 프로그램 코드(기계어)와 데이터들을 자유롭게 주기억장치에 저장한 후 프로그램 코드를 저장된 순서대로 차례로 실행
- 실행되는 프로그램 코드의 주소는 CPU의 내부 레지스터인 프로그램 카운터(PC: Program Counter)에 의해 자동으로 변경됨

### 2) 폰 노이만 구조 - 2

- 근래의 컴퓨터 구조의 근간을 이루는 기본 구조로 **다음과 같은 주요 구성으로 이루어진 컴퓨터 구조** (IAS 컴퓨터에 적용)
  1. 프로그램 제어 유니트 (명령어 인출/해독)
  2. 산술논리연산장치(ALU)
  3. 주기억장치 : 명령어와 데이터를 모두 저장
  4. 입출력장치

### 3) 폰 노이만 구조 – 3

- 시스템 버스의 최초 구성방식으로 하버드 구조의 반대 개념(향후 학습)



# 1.4 컴퓨터 구조의 발전 과정



p29~p34

## ■ IAS 컴퓨터

✓ 1946년~1952년: 폰 노이만이 IAS 연구소에서 개발

➢ 폰 노이만: 프린스턴 대학 및 IAS(Institute for Advanced Studies)의 수학 교수

✓ **프로그램 저장과 변경이 가능한 최초의 디지털 컴퓨터**

✓ **근래의 컴퓨터 구조와 동일한 구조 → 폰 노이만 구조**

✓ 주요 구성요소

① 프로그램 제어 유니트(Program Control Unit)

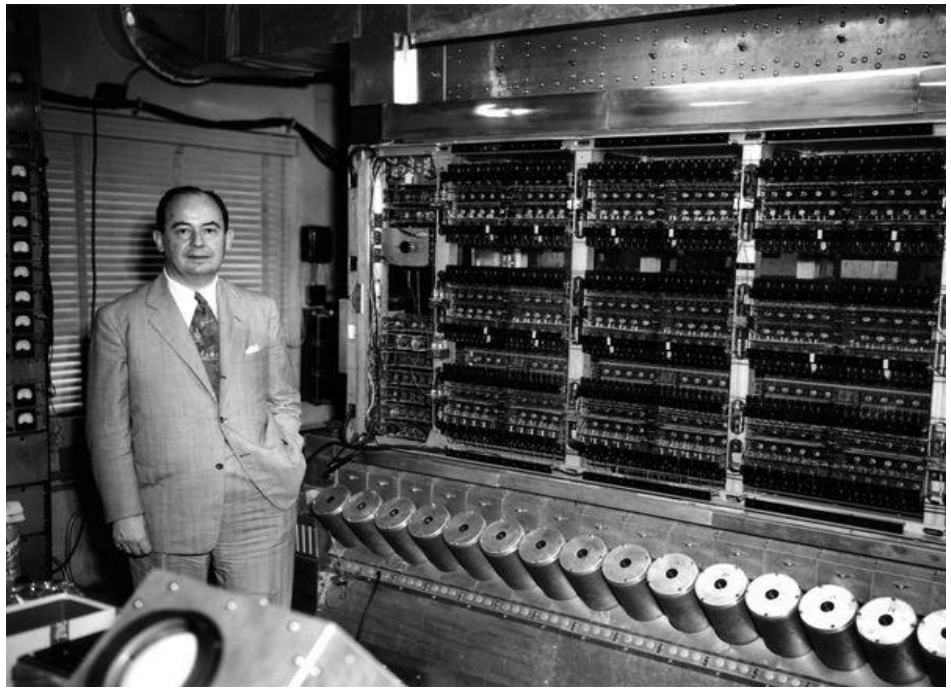
❖ 명령어 인출/해독

② 산술논리연산장치(ALU)

③ 주기억장치

❖ 명령어와 데이터를 모두 저장

④ 입출력장치



참고: IAS 컴퓨터 직전에 만들어진 EDVAC 컴퓨터 역시 폰 노이만 구조를 사용했으나 이 컴퓨터는 제어장치, 연산장치가 분리된 개념이며 실제로 이들이 통합되어 CPU라는 명칭으로 폰 노이만 구조를 완벽하게 최초로 구현된 제품은 IAS 컴퓨터임 → IAS 컴퓨터가 최근 컴퓨터의 기원이 됨

# 1.4 컴퓨터 구조의 발전 과정

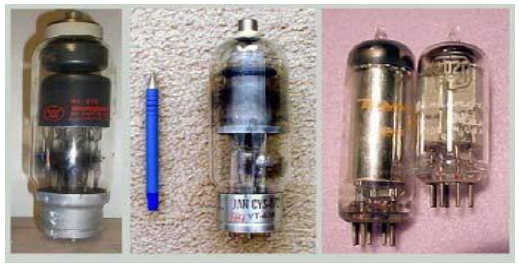


## ■ 주요 컴퓨터 부품들의 발전 과정

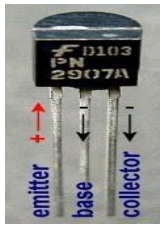
- ✓ 릴레이(relay) → 진공관 → 트랜지스터(Tr) → 반도체 집적회로(IC)
- ✓ 발전 과정에서 개선된 특성들
  1) 처리속도 향상
  2) 저장용량 증가
  3) 크기 감소
  4) 가격 하락
  5) 신뢰도 향상
- ✓ 초기 컴퓨터들의 근본적인 설계 개념과 동작 원리가 현대의 컴퓨터들과 거의 같음



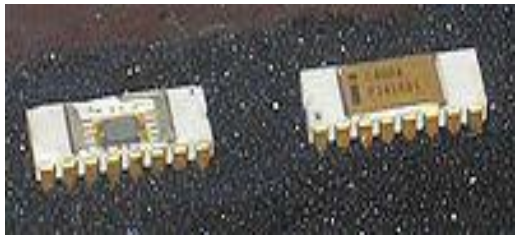
< 릴레이 >



< 진공관 >




< 트랜지스터 >



< Integrated Circuit >

# 1.4 컴퓨터 구조의 발전 과정

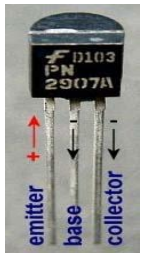


p34~p38

## ■ 주요 컴퓨터 부품들의 발전 경위

### ✓ 트랜지스터(Transistor=Tr)

- 초기(제1세대) 전자식 컴퓨터의 핵심 부품인 진공관을 대체한 전자 부품
- 진공관보다 작고 싸며 더 적은 열을 발산
- 반도체 재료인 실리콘(Si)으로 만들어진 고체(solid-state) 장치
- 제2세대 컴퓨터들의 핵심 부품
- 초기 컴퓨터들은 약 1000개의 트랜지스터들로 구성



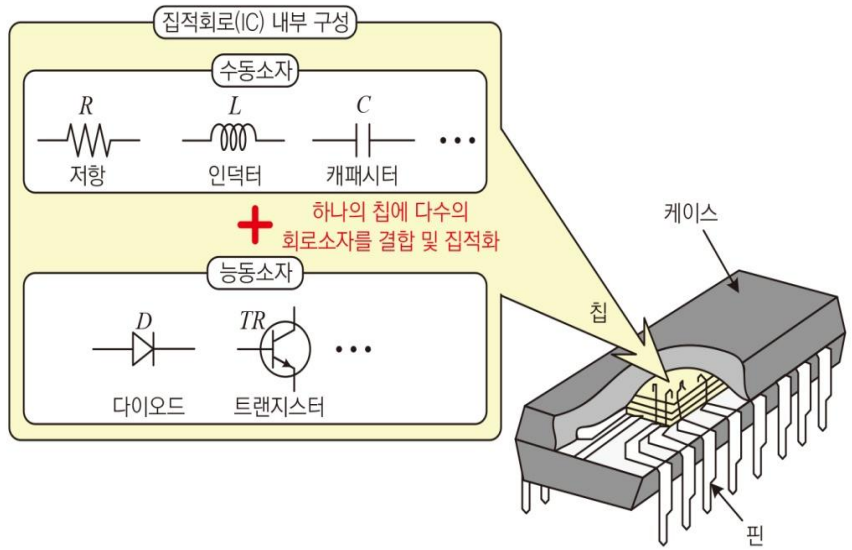

< 트랜지스터 >

### ✓ 집적 회로(Integrated Circuit: IC)

- 수만 개 이상의 트랜지스터들을 하나의 반도체 칩에 집적시킨 전자 부품
- 제3세대 이후 컴퓨터들의 핵심 부품



< Integrated Circuit >



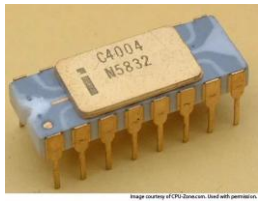
# 1.4 컴퓨터 구조의 발전 과정



p34~p38

## ■ 집적도에 따른 IC의 분류

1) SSI(Small Scale IC)
   - ➢ 수십 개의 트랜지스터들이 집적되는 소규모 IC
   - ➢ 최근에는 주로 기본적인 디지털 게이트(digital gate)들을 포함하는 칩으로만 사용됨
2) MSI(Medium Scale IC)
   - ➢ 수백 개의 트랜지스터들이 집적되는 IC
   - ➢ 카운터(counter), 해독기(decoder) 또는 시프트 레지스터(shift register)와 같은 조합 회로나 순차 회로를 포함하는 칩
3) LSI(Large Scale IC)
   - ➢ 수천 개의 트랜지스터들이 집적되는 대규모 IC
   - ➢ 8비트 마이크로프로세서 칩이나 소규모 반도체 기억장치 칩




인텔 4004 CPU: 세계 최초의 민간 용으로 출시된 단일 칩 4비트 마이크로프로세서

4) VLSI(Very Large Scale IC)
   - ➢ 수만 내지 수십만 개 이상의 트랜지스터들이 집적되는 초대규모 IC
   - ➢ 마이크로프로세서 칩들과 대용량 반도체 기억장치 칩
5) ULSI(Ultra Large Scale IC)
   - ➢ 수백만 개 이상의 트랜지스터들이 집적되는 32비트급 이상 마이크로프로세서 칩들과 수백 메가비트 이상의 반도체 기억장치 칩들 및 앞으로 출현할 고밀도 반도체 칩들을 지칭하기 위한 용어로서, VVLSI(Very VLSI)라고도 불림

# 1.4 컴퓨터 구조의 발전 과정



p34~p38

## ■ IC 사용에 따른 이점

1) 전기적 통로가 짧아짐(배선의 저항 감소) → 동작 속도가 크게 상승
2) 컴퓨터 크기의 감소
3) 칩 내부의 회로들 간의 상호연결 → 부품들의 신뢰도 향상
4) 전력소모 감소 및 냉각장치의 소형화
5) 컴퓨터 가격 하락

## ■ LSI 기술로 부터 마이크로 프로세서가 탄생

- ✓ LSI 기술이 적용된 컴퓨터를 4세대 컴퓨터로 분류
- ✓ 개인용 컴퓨터가 출현했고, 초고속 슈퍼컴퓨터의 개발이 본격화
- ✓ 이와 같은 집적화에 따라 IC 패키징 기술이 발달되기 시작

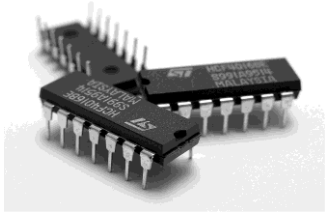
# 1.4 컴퓨터 구조의 발전 과정



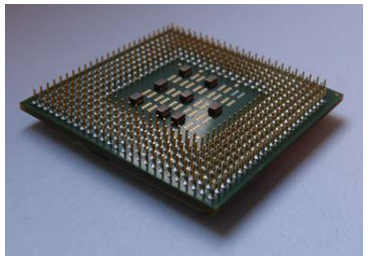
p34~p38

## ■ IC 칩 패키징 기술 발전

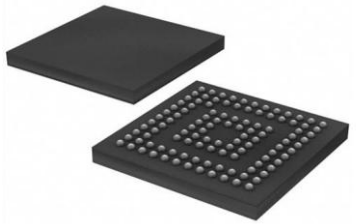
- ✓ DIP(dual in-line package)
- ✓ PGA(pin grid array)
- ✓ BGA(ball grid array)
- ✓ 2.5D 및 3D IC 패키징 기술 개발
  - ➢ 부품의 소형화, 고성능화, 융복합화 지원
  - ➢ 2.5D IC
    - ❖ 하나의 패키지 기판 상에 여러 종류의 칩들을 수평으로 배치
    - ❖ 칩들 간 신호선 접속은 인터포저 층(interposer layer) 이용
  - ➢ 3D IC
    - ❖ 여러 종류의 칩들을 수직으로 쌓아 올려 면적 최소화
    - ❖ HBM(high bandwidth memory)도 이 기술을 이용 <8.5절 참조>



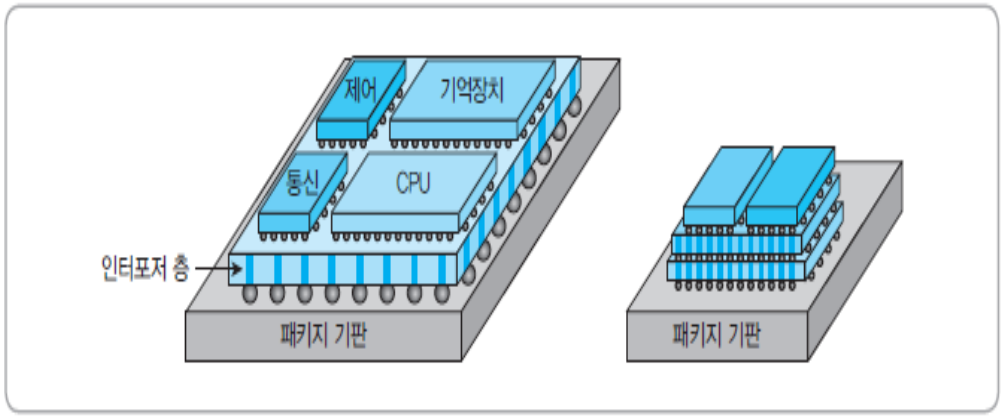
DIP



PGA



BGA




(a) 2.5D IC (b) 3D IC

# 1.4 컴퓨터 구조의 발전 과정



## ■ 추가: 주요 핵심부품에 세대별 분류 방법

교재에는 없으나 이러한 분류법도 많이 사용

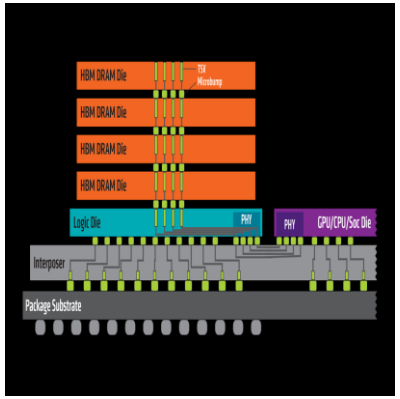
✓ 주로 사용되는 전자회로(논리회로)로 세대를 구분한다

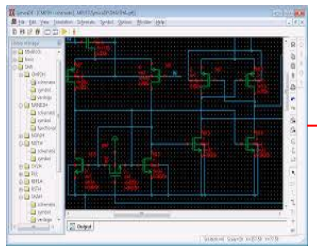
- 제 1세대(1951~1958)
  - ❖ 데이터의 저장과 처리에 진공관을 사용
- 제 2세대(1958~1963)
  - ❖ 회로소자로 트랜지스터를 사용
- 제 3세대(1964~1970)
  - ❖ 컴퓨터에 집적회로 IC(SSI, MSI)를 사용
- 제 4세대(1971~ )
  - ❖ 고밀도 집적 회로(LSI)를 사용 → 마이크로프로세서의 등장
- 제 5세대(진행중)
  - ❖ 초고밀도 집적 회로(VLSI) 이상을 사용

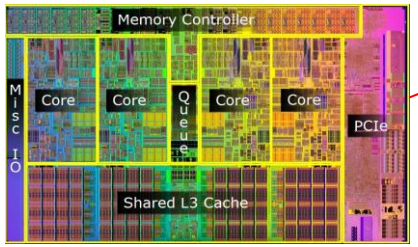
# 1.4 컴퓨터 구조의 발전 과정

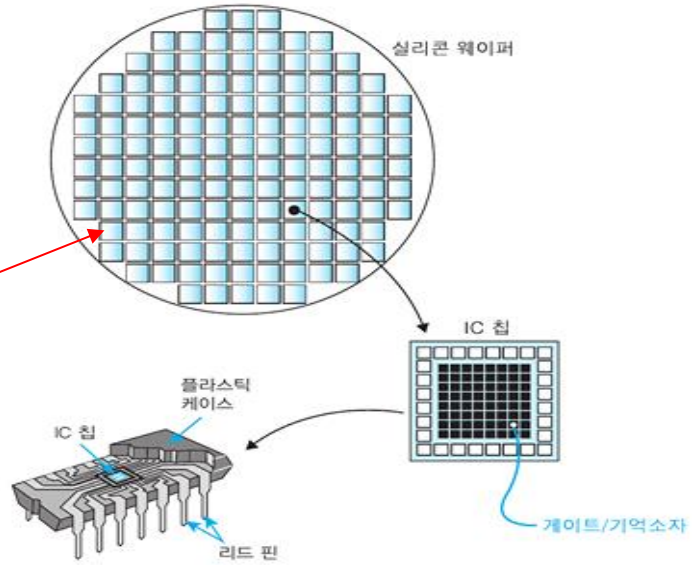
## ■ 반도체 기술의 발달 – 집적도, 고속화, 고신뢰성

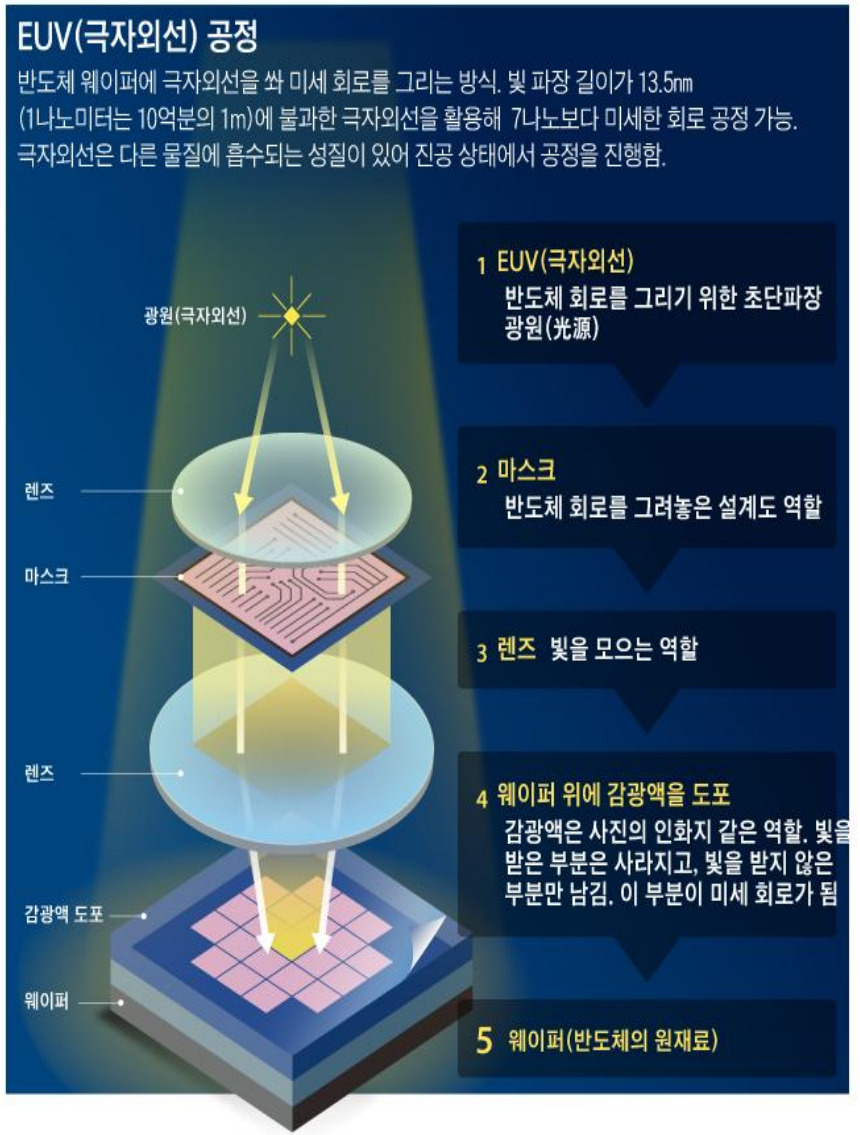
- ✓ 주요 핵심 기술
  1) 미세공정 → 2nm 공정
  2) 적층 구조
  3) 패키징(Packaging) 기술
  4) 웨이퍼 크기의 증가 → 12인치(300mm)
  5) 공정기술 → 수율(Yield) 등
- ✓ 소형화, 고속, 신뢰도 상승, 가격하락





# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조

## ■ 프로세서 구조의 다양화 – 칩의 통합도에 따른 CPU 분류

MPU = 마이크로 프로세서(CPU)
MCU = 마이크로 컨트롤러 = CPU + 주변기기 일체화

1) MPU (Micro-Processor Unit, 마이크로 프로세서) ≒ CPU
   - ➢ CPU 기능을 가진 (몇 개의) 반도체 칩을 하나의 작은 실리콘 칩에 집적한 처리장치
   - ➢ 인텔 ×86 계열, Power PC(모토롤라+IBM) 계열, 미국 선사의 SPARC 계열 등
2) MCU (Micro-Controller Unit, 마이크로 컨트롤러)
   - ➢ 단일 칩 마이크로컴퓨터(One-Chip 또는 Single-Chip Microcomputer)라고도 부름
   - ➢ CPU 기능, ROM, RAM, 입출력 제어 인터페이스 회로 등을 한개의 칩 내부에 내장
   - ➢ 범용의 목적보다는 기기 제어용 임베디드 시스템에 주로 사용
   - ➢ 인텔 8051, 마이크로칩 PIC 계열, ATMEL AVR 계열, ARM 계열, ESP 계열 등
3) SoC (System on a Chip)
   - ➢ MCU보다 한 차원 높은 수준으로, 고성능 MPU와 주변의 필수 칩들을 하나로 합쳐 놓은 차원이 다른 칩
     - ❖ 고성능 MPU(CPU) + GPU + 통신칩(Wi-Fi) + I/O + 기타 조수들 (NPU, DSP 등)
   - ➢ 라즈베리 파이 칩 (Broadcom), Apple M-시리즈, 스마트폰 AP
4) AP (Application Processor)
   - ➢ 단순 기기 제어용인 MCU 수준을 넘어, OS 및 복잡한 애플리케이션을 구동하기 위해 고성능 CPU 코어(주로 ARM 계열)와 다양한 제어/연산 칩을 통합한 특화된 SoC → 모바일 기기(스마트폰, 태블릿 등) 및 스마트 디바이스에 특화
   - ➢ 삼성전자 Exynos, 애플 A-series, 퀄컴 Snapdragon, 엔비디아 Tegra, 인텔 Atom, 프리스케일 i.MX

# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조



CPU GPU NPU TPU (01m44s)

## ■ 프로세서 구조의 다양화 – 특수 목적 프로세서

✓ 범용 프로세서가 모든 일을 하면 비효율적이기 때문에 CPU가 특정 분야의 복잡한 계산을 맡기는 보조 프로세서 칩이 필요

1) GPU (Graphic Processing Unit) → 8장 상세 학습
   - ➢ 수치연산 전용으로 개발된 SIMD 프로세서 → 수천 개의 작은 코어로 3D 그래픽이나 단순 반복 계산을 동시에(병렬로) 처리
   - ➢ 그래픽 카드용으로 개발되었으나 최근에는 복잡한 연산등에 매우 적극적으로 활용 → GPGPU
2) DSP (디지털 신호 처리 장치)
   - ➢ 소리(오디오), 영상, 통신 전파 같은 아날로그 신호를 실시간으로 디지털로 변환하고 처리하는 데 특화
   - ➢ 음성 인식, 노이즈 캔슬링, 5G 통신 모뎀
3) NPU / TPU (Neural / Tensor Processing Unit. 신경망 처리 장치)
   - ➢ AI 모델(신경망)이 주로 하는 '행렬 곱셈 연산'을 저전력으로 매우 빠르게 처리하도록 설계
   - ➢ NPU(Neural PU)는 AI 기능(학습+추론) 중에서 주로 추론기능에 집중한 온디바이스 칩
   - ➢ TPU(Tensor PU)는 구글이 개발한 NPU의 브랜드 이름이며 학습+추론 모두를 균형 있게 담당 (클라우드/데이터센터 환경 특성)
4) NPU (Network Processing Unit)
   - ➢ 네트워크 장비에 특화된 전용 프로세서 → CPU가 처리하기에는 너무 많은 네트워크 데이터 패킷(Packet)을 고속으로 처리
     - ❖ 패킷 분류 및 라우팅 (경로 설정), QoS (서비스 품질 보장), 방화벽 (DPI - 심층 패킷 검사)
   - ➢ 시스코(Cisco), 주니퍼(Juniper) 등의 고성능 라우터, 스위치, 통신 장비

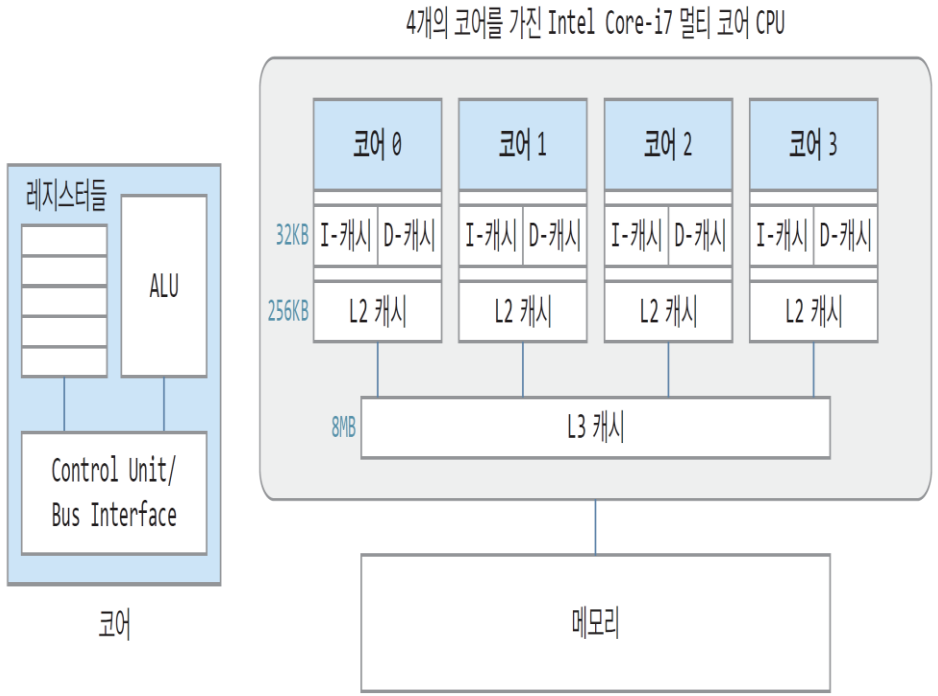
# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조



## ■ 멀티 코어 (다중 코어)기술 → 8장 상세 학습

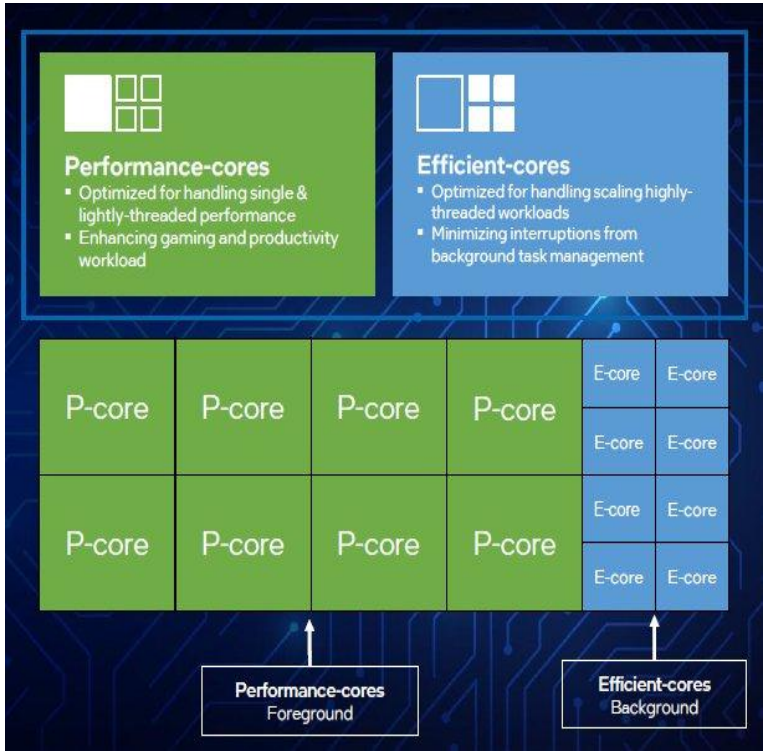
✓ **하나의 CPU 칩**(Die) 내부에 **두개 이상의 코어**를 집적 시켜서 CPU의 성능을 향상

➢ **Core(코어)**: CPU의 기본 기능을 담당하는 회로 모듈 (제어 유니트 + ALU + 레지스터)



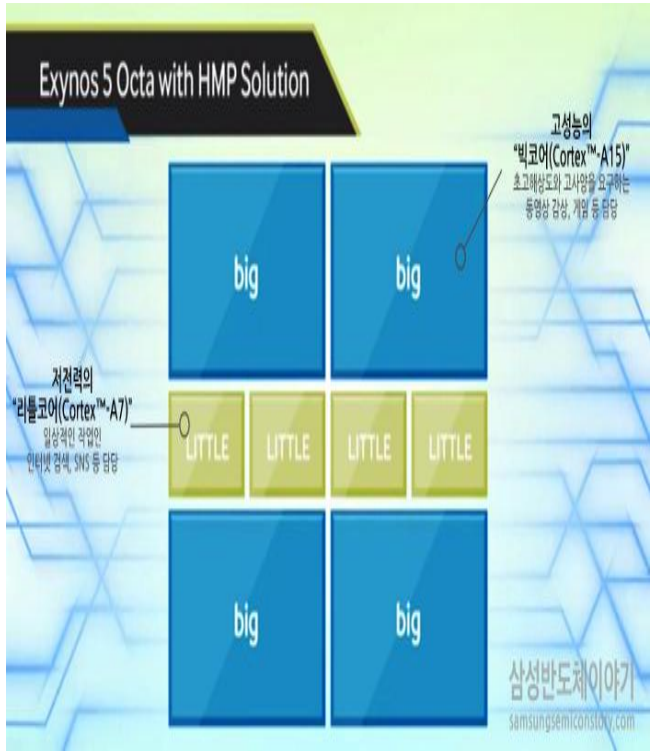

(a) 코어의 내부 구조 (b) 4개의 코어를 가진 Intel Core-i7 멀티 코어 CPU 사례

인텔도 12세대 CPU 부터는 big.LITTLE 코어 구조로 변경




< 인텔 12세대 이후 CPU: P Core, E core >

ARM의 big.LITTLE 코어 구조



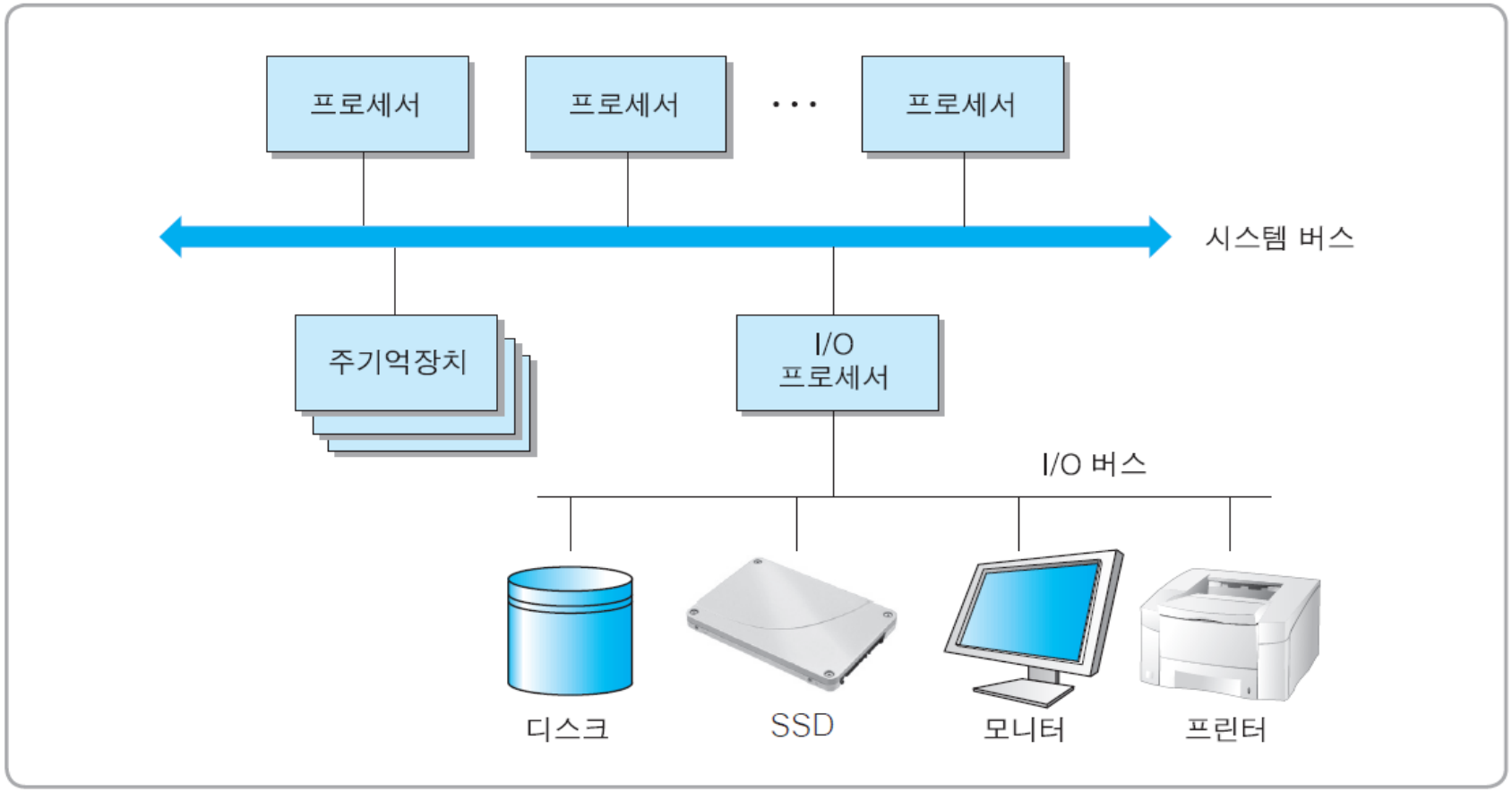

< 삼성전자 엑시노드 AP >

# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조



## ■ 다중프로세서 시스템의 구조 → 8장 상세 학습

- ✓ 여러 개의 CPU가 **하나의 시스템 버스**에 연결되어 있는 구조
- ✓ 하나의 일을 나누어 실행하거나 여러 개의 일을 각각 수행 가능



# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조



## ■ AP(Application Processor)

✓ 특별한 목적(AP)으로 IC 칩 안에 다양한 기능을 집약한 **특수용도의 사용을 목적으로 설계된 CPU**

➢ 삼성전자의 엑시노스(Exynos) 삼성 Exynos 9 (8895)

❖ 삼성 노트8, 갤럭시 S8, S8+에 사용

❖ 현재는 Exynos 2500(3nm 공정, 갤럭시 Z플립 7에 적용)이 최신 제품



# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조



## ■ 하버드 구조

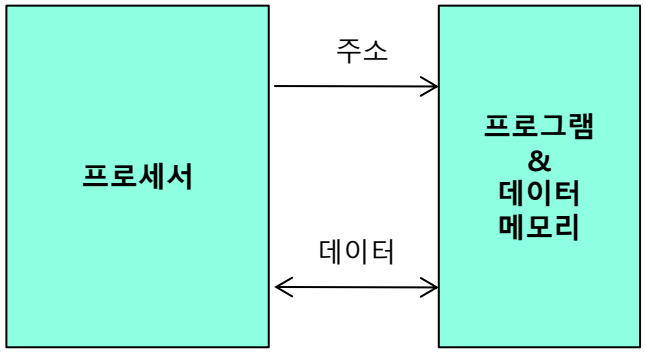
교재에는 없으나 매우 중요한 개념

### 1) 폰 노이만 구조

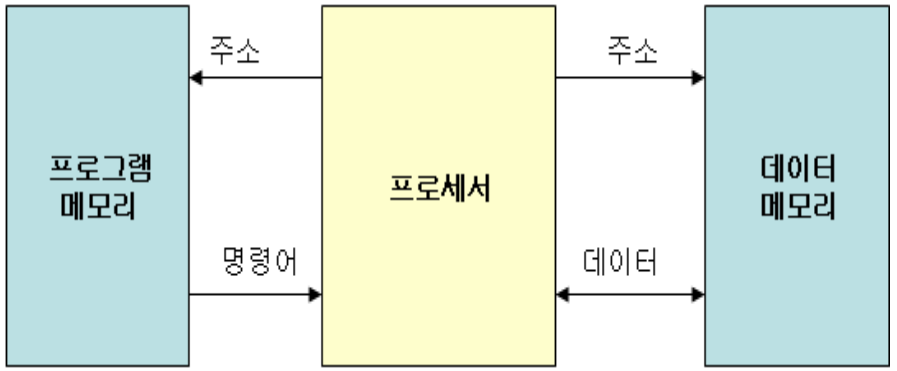
- 프로그램과 데이터는 같은 메모리 내에 저장 → **하나의 시스템 버스를 공통으로 사용**
- 프로그램과 데이터는 동일한 버스를 통해 접근(Access) → 버스 병목현상 초래

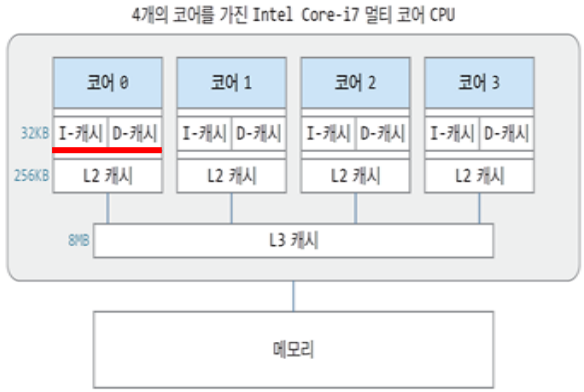
### 2) 하버드 구조

- 프로그램과 데이터가 각기 다른 메모리에 저장되는 컴퓨터 구조 → **시스템 버스의 분리**
- 고속 메모리 입출력 가능 → 폰 노이만 구조의 병목현상 개선
- 외부 핀이 많아지는 단점이 있었으나 반도체 집적 기술의 발달로 근래의 대부분의 컴퓨터에서 사용(캐시메모리 이용)
- 특히 MCU는 메모리를 내장하고 있기 때문에 메모리용 외부 버스를 사용하지 않기 때문에 대부분 하버드 구조를 적용




< 폰 노이만 구조 >




< 하버드 구조 >

# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조



## ■ RISC 프로세서 → 2장 상세 학습

### 1) CISC (Complex Instruction Set Computer) 프로세서

- 복합적인 연산들을 처리하는 **다양한 종류의 명령어(기계어)**들을 가진 기존의 프로세서 → 초기 CPU
- CISC의 문제점
  - ① 명령어 비트 수가 길어져서 기억장치로부터 읽어오는데 여러 번의 인출 필요
  - ② 다양한 종류와 길이의 명령어를 해독하기 위한 제어 유니트(제어장치)의 내부 회로 복잡도 증가
  - ③ 명령어 해독 시간 증가 (매크로 명령어를 이용해 해당되는 마이크로 명령어 루틴의 시작주소를 찾아야 함)
  - ④ 하나의 (매크로) 명령어를 위해 CPU 내부적으로 복수개의 마이크로 명령어들을 순차적으로 처리해야 하기 때문에 실행시간이 증가

### 2) RISC (Reduced Instruction Set Computer) 프로세서 → 2장 상세 학습

- CISC의 한계성을 극복하고자 제안한 새로운 프로세서 구조
- **간략화 된 소수의 명령어들**만 지원하며 주로 고정배선 제어 유니트를 활용
- 고정된 길이의 명령어, 많은 레지스터들(주기억장치 접근 최소화)를 포함
- 빠른 실행속도를 지원하나 명령어가 제한적이어서 컴파일러가 매우 중요

근래에는 순수한 RISC/CISC CPU는 거의 없다. 서로의 장단점을 섞어 사용한다.

# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조

## ■ 명령어/데이터의 동시 처리 기술

1) Pipelining (파이프라이닝) → 2장 학습

   ➢ CPU 내부에서 명령어의 수행 단계를 여러 단계로 나누어 동시에 여러 개의 명령어를 처리 → 속도 향상

      ❖ 인출, 해독, 실행, 저장 등을 동시에 실행

2) Superscalar (슈퍼스칼라) → 2장 학습

   ➢ CPU 내부에 두 개 혹은 그 이상의 명령어 파이프라인을 포함시킨 구조 → 속도 향상

   ➢ 두개의 명령어가 동시에 인출되고 처리되는 것이 2-way 슈퍼스칼라

3) RAID – 추가자료-보조기억장치 → 6장 학습

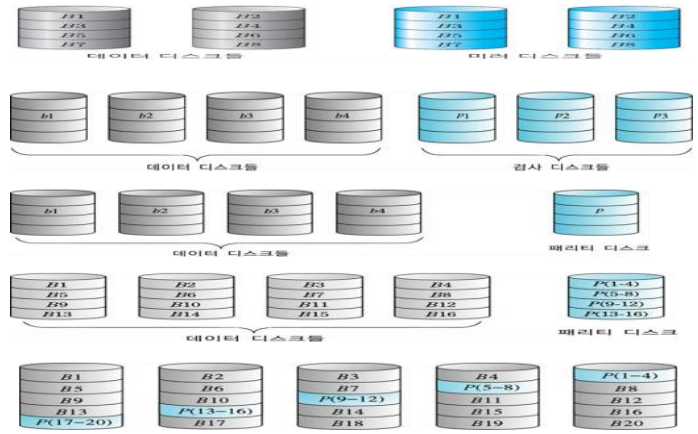
   ➢ 여러 개의 하드 디스크를 하나의 논리 디스크로 사용하는 기술

   ➢ 저장장치의 신뢰성을 높이거나 전체적인 성능을 향상시키는 등의 다양한 목적





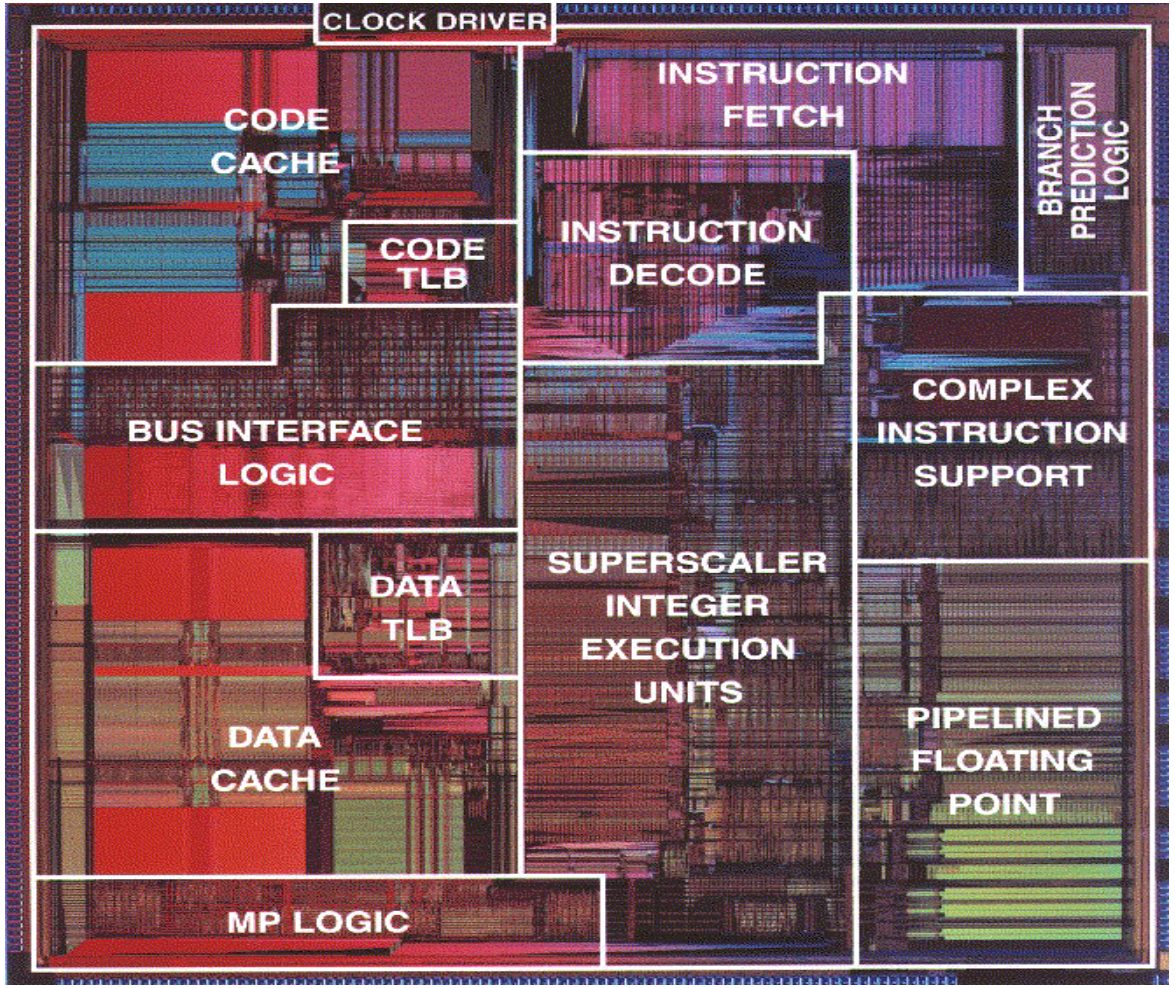




# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조



## ■ 인텔 Pentium(32bit) 프로세서 Die-level Layout

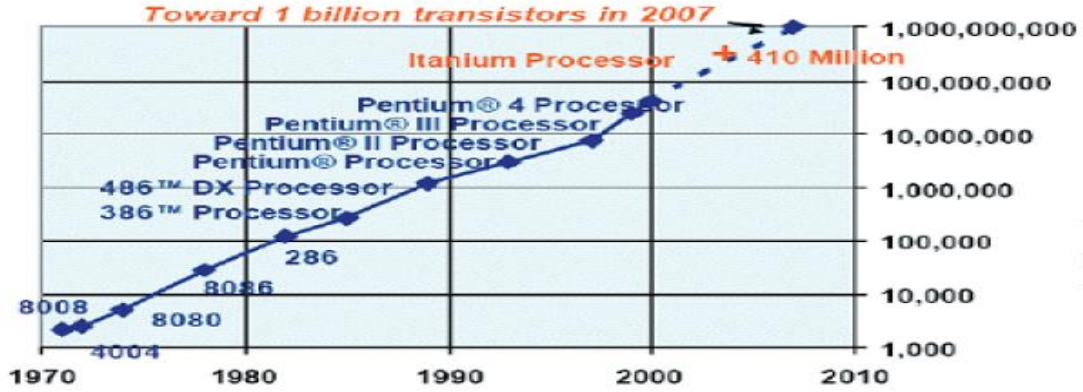


# 추가: 컴퓨터 시스템에 영향을 끼친 주요 기술 및 구조

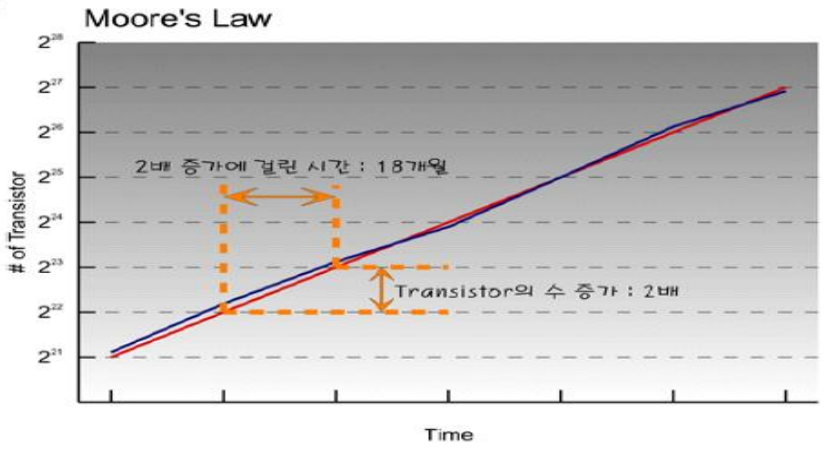
## ■ 무어의 법칙 (Moore's Law)

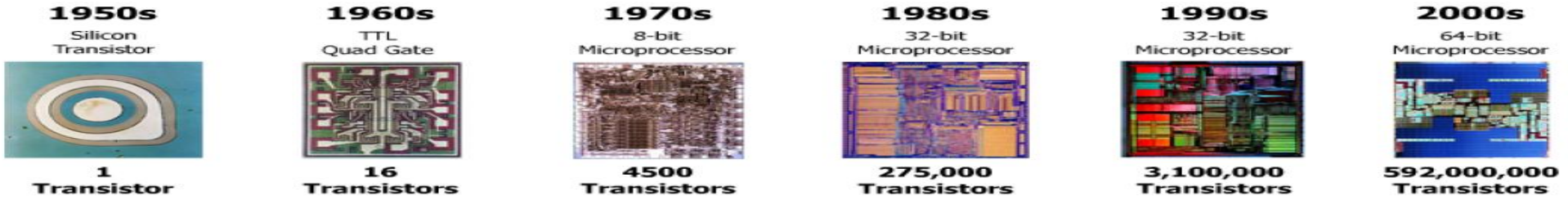
교재에는 없으나 매우 중요한 개념

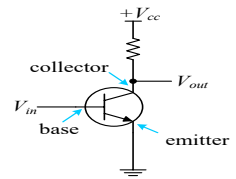
✓ **동일 면적의 반도체 칩에 집적되는 트랜지스터 개수가 18-24개월 마다 2배 씩 증가한다**



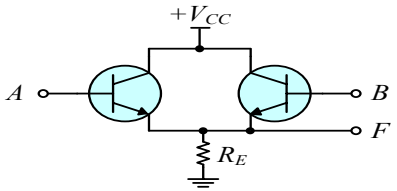

반도체 소자의 집적도 향상을 예상하는 무어의 법칙



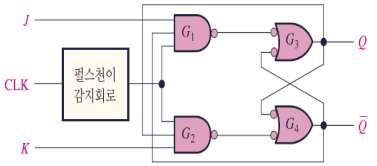



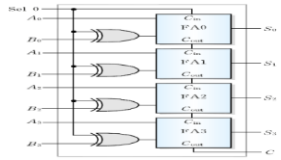

트랜지스터



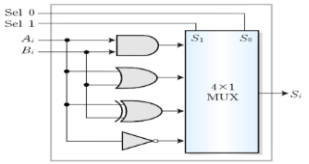

OR 게이트




JK F/F



ALU : 산술연산




ALU : 논리연산

# 1.4 컴퓨터 구조의 발전 과정



p38~p50

## ■ 컴퓨터시스템의 분류와 발전 동향

### 1) 개인용 컴퓨터(Personal Computer)

✓ 특징

- 소형, 저가, 수십년 전의 대형 메인프레임 컴퓨터의 성능을 능가

✓ 주요 발전 동향

- 성능이 개선된 새로운 마이크로프로세서(**MPU**)들의 등장에 따라 PC의 성능은 계속 향상
  - 8, 16, 32, 64 비트 단위의 데이터 처리 및 기억장치 주소 사용 PC들도 출시 중
  - 하나의 CPU가 복수 명령어 실행 유니트 또는 코어들을 포함하는 슈퍼스칼라, 멀티-코어 구조로 발전
  - 칩의 집적도가 높아지면서 주변 요소들이 CPU 칩 내부에 집적함에 따라, 속도와 신뢰도의 향상
- GPU(Graphic Processing Unit)를 계산보조장치로 사용함으로써 고속 그래픽 처리 뿐 아니라 복잡한 과학기술 계산들도 높은 속도로 처리할 수 있게 됨
- 주기억장치와 보조저장장치의 용량 증가, 종류 다양화

✓ 종류(유형)

- 데스크탑 컴퓨터, 노트북 컴퓨터, 넷북(netbook), 태블릿(tablet) PC, 포켓(pocket) PC 등

# 1.4 컴퓨터 구조의 발전 과정

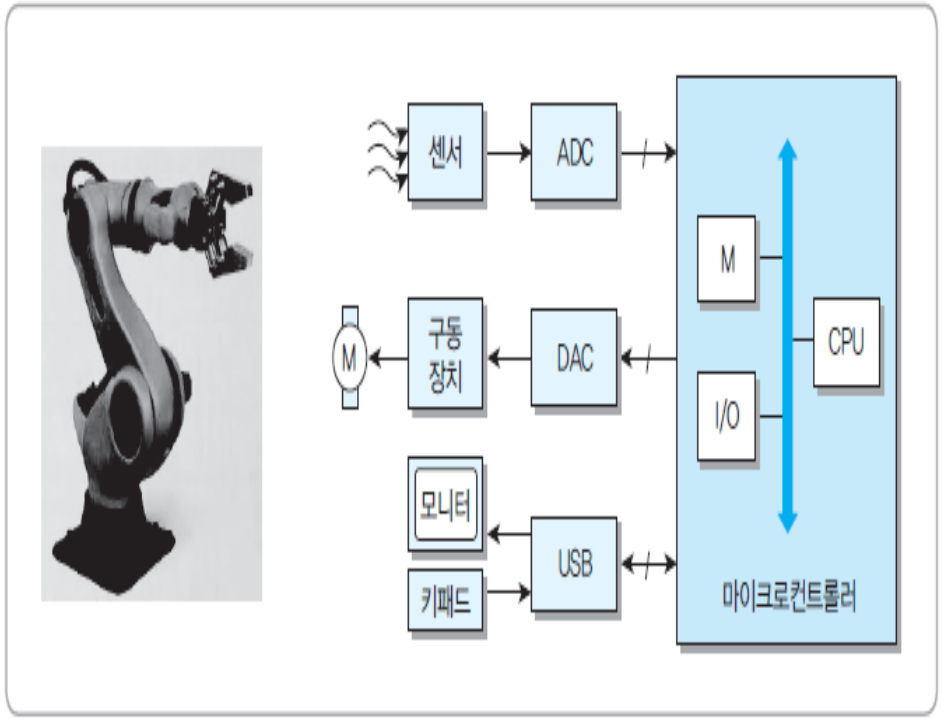
p38~p50

## ■ 컴퓨터시스템의 분류와 발전 동향

### 2) 임베디드 컴퓨터

- Embedded Computer(내장 컴퓨터라고도 부름)
- 기계 장치나 전자 장치들의 내부에 포함되어, 그 장치들의 동작을 제어(control)하는 컴퓨터들
- [예] 가전제품, 컴퓨터 주변기기, 모바일폰, 비디오 게임기 등
- 8비트 마이크로컨트롤러(micro-controller) 를 이용한 초소형부터 64비트 컴퓨터에 이르기까지 다양
  - ❖ 임베디드 컴퓨터에서는 **MCU**(Micro Controller Unit)를 주로 사용
- 최소의 비용으로, 필요한 만큼의 성능 제공
- 실시간 처리(real-time processing)
- IoT(Internet of Things) 및 지능형 로봇의 핵심 요소로 사용되어 더욱 다양해지고 보급도 확대될 전망

근래에는 64비트 MCU도 많이 사용




(a) 로봇 팔 (b) 임베디드시스템 구성도

# 1.4 컴퓨터 구조의 발전 과정

p38~p50

## ■ 컴퓨터시스템의 분류와 발전 동향

### 3) 서버급 컴퓨터시스템

- ➢ 워크스테이션(Workstation)
  - ❖ CPU: 64비트 마이크로프로세서 사용
  - ❖ 고속 그래픽 처리 하드웨어 포함
  - ❖ 주요 응용: 3차원 동영상처리, 시뮬레이션, 컴퓨터 이용 설계(CAD) 등
  - ❖ OS: UNIX, LINUX
- ➢ 슈퍼 미니컴퓨터(Super-minicomputer)
  - ❖ 시스템 구조: 다중프로세서(multiprocessor) 구조
  - ❖ CPU의 수: 20 ~ 30 의 64bit CPU
  - ❖ 성능: VAX-11 미니컴퓨터 성능의 수십 배 이상
  - ❖ OS: UNIX
  - ❖ 서버(server)급 시스템의 다운사이징(downsizing)화 주도
    - » 네트워크에 접속된 다수의 중형급 컴퓨터 시스템들을 응용(혹은 용도)별로 구분하여 사용하는 컴퓨팅 환경이 가능해지게 함

# 1.4 컴퓨터 구조의 발전 과정

p38~p50

## ■ 컴퓨터시스템의 분류와 발전 동향

### 4) 대형 메인프레임 컴퓨터(Mainframe computer)

- 중앙집중식 컴퓨팅을 위한 대형 컴퓨터
- IBM 360 및 370 계열, 3081, 3090 등으로 계속 발전하였으며, 최근 시스템 보안 및 통신 기능이 대폭 보강된 IBM Enterprise 계열 출현
- 다중 I/O 채널을 이용한 고속 I/O 처리 능력 보유
- 대용량 저장장치 보유 → 대규모 데이터베이스 저장 및 관리용으로 사용
- 정부기관, 은행, 대형 인터넷 포탈사이트 등에서 대규모 데이터베이스(빅데이터) 저장/관리용으로 사용

### 5) 슈퍼컴퓨터(Supercomputer)

- 처리 속도와 기억장치 용량이 다른 컴퓨터들에 비하여 상대적으로 월등한 컴퓨터 시스템들
- 분류 기준이 계속적으로 상승
  - 최초의 슈퍼컴퓨터인 CRAY-1의 속도는 100 MFLOPS(메가 플롭스) → 최근 PFLOPS(페타 플롭스)급 (수백만 배 향상)
- 주요 응용 분야들 → 향후 양자컴퓨터가 대신할 분야 
  - VLSI 회로 설계, 항공우주공학, 천문학(일기 예보), 구조 공학, 유전 탐사, 핵공학, 인공지능, 입체 영상처리 등과 같은 대규모 과학 계산 및 시뮬레이션

# 1.4 컴퓨터 구조의 발전 과정



## ■ 참고: 수의 단위

| $10^n$ | 접두어 | 기호 | 배수 | 십진수 |
|---|---|---|---|---|
| $10^{24}$ | 요타 (yotta) | Y | 자 | 1 000 000 000 000 000 000 000 000 |
| $10^{21}$ | 제타 (zetta) | Z | 십해 | 1 000 000 000 000 000 000 000 |
| $10^{18}$ | 엑사 (exa) | E | 백경 | 1 000 000 000 000 000 000 |
| $10^{15}$ | 페타 (peta) | P | 천조 | 1 000 000 000 000 000 |
| $10^{12}$ | 테라 (tera) | T | 조 | 1 000 000 000 000 |
| $10^{9}$ | 기가 (giga) | G | 십억 | 1 000 000 000 |
| $10^{6}$ | 메가 (mega) | M | 백만 | 1 000 000 |
| $10^{3}$ | 킬로 (kilo) | k | 천 | 1 000 |
| $10^{2}$ | 헥토 (hecto) | h | 백 | 100 |
| $10^{1}$ | 데카 (deca) | da | 십 | 10 |
| $10^{0}$ | | | 일 | 1 |
| $10^{-1}$ | 데시 (deci) | d | 십분의 일 | 0.1 |
| $10^{-2}$ | 센티 (centi) | c | 백분의 일 | 0.01 |
| $10^{-3}$ | 밀리 (milli) | m | 천분의 일 | 0.001 |
| $10^{-6}$ | 마이크로 (micro) | μ | 백만분의 일 | 0.000 001 |
| $10^{-9}$ | 나노 (nano) | n | 십억분의 일 | 0.000 000 001 |
| $10^{-12}$ | 피코 (pico) | p | 일조분의 일 | 0.000 000 000 001 |
| $10^{-15}$ | 펨토 (femto) | f | 천조분의 일 | 0.000 000 000 000 001 |
| $10^{-18}$ | 아토 (atto) | a | 백경분의 일 | 0.000 000 000 000 000 001 |
| $10^{-21}$ | 젭토 (zepto) | z | 십해분의 일 | 0.000 000 000 000 000 000 001 |
| $10^{-24}$ | 욕토 (yocto) | y | 일자분의 일 | 0.000 000 000 000 000 000 000 001 |

# 1.4 컴퓨터 구조의 발전 과정

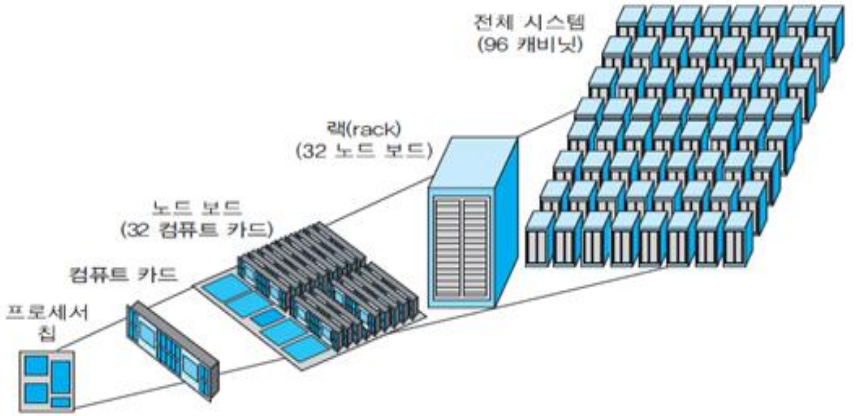


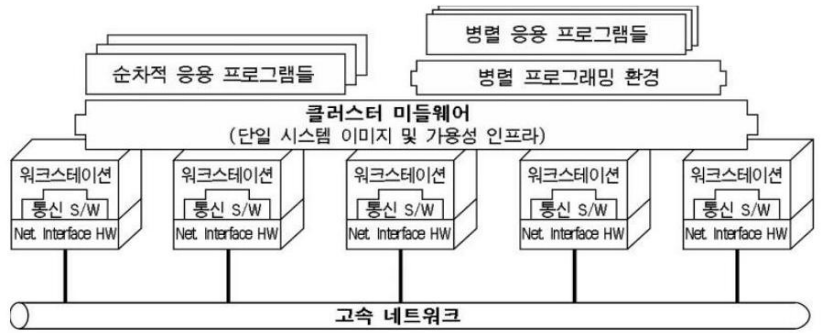
p38~p50

## ■ 구조적 특징에 따른 슈퍼컴퓨터의 분류 → 8장 상세 학습

1) 파이프라인 슈퍼컴퓨터(Pipeline Supercomputer)
   - ➢ 복잡한 초고속 연산회로들이 포함된 적은 수의 CPU들로 구성
   - ➢ 초기의 슈퍼컴퓨터 구조 → CRAY Y-MP, CRAY-2, VPP500 등
   - ➢ 슈퍼파이프라인 구조를 이용하여 고속 벡터 계산 수행
2) 대규모 병렬컴퓨터(Massively Parallel Computer: MPP)
   - ➢ 하나의 큰 작업을 분담하여 동시에 처리하는 병렬처리 기술 이용
   - ➢ 상호 연결망을 이용해 수백 혹은 수천 개 이상의 범용 프로세서들로 구성
   - ➢ 시스템 사례 : IBM BlueGene/Q 슈퍼컴퓨터
3) 클러스터 컴퓨터(Cluster Computer)
   - ➢ 고속 LAN으로 연결된 PC들 혹은 워크스테이션들의 집합체
   - ➢ 클러스터 미들웨어를 이용하여 단위 컴퓨터들에 포함된 모든 자원들을 단일 시스템 이미지(Single System Image: SSI)로 통합
   - ➢ 저렴한 가격으로 고성능, 고신뢰성을 갖는 병렬컴퓨팅 환경 구축 가능

발전 순서
파이프라인 슈퍼컴퓨터 → 대규모 병렬컴퓨터 → 클러스터 컴퓨터



# 1.4 컴퓨터 구조의 발전 과정



p38~p50

## ■ 슈퍼컴퓨터 구조의 세부적 발전 동향 – 1/2

✓ CPU-GPU 결합

➢ CPU가 GPU & TPU(Tensor Processing Unit) 와 결합되는 혼합형 구조(heterogeneous architecture)

✓ 모듈식 설계

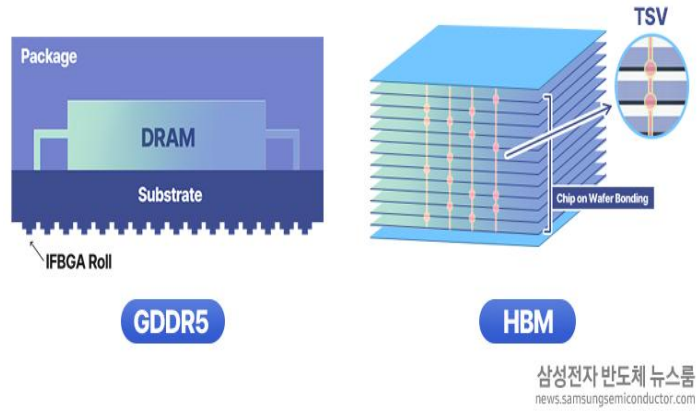
➢ 노드(node) 부품 교환으로 시스템 전체 성능 향상 가능

✓ 고속 상호연결망 개발

➢ 고대역폭 저지연 상호연결(high-bandwidth low-latency interconnection)

❖ [예] CXL(Compute Express Link), 광 연결망(optical interconnect)

✓ 고대역폭 기억장치(HBM)




➢ 대형 언어 모델(LLM)과 같은 대규모 데이터 세트들을 신속히 액세스 위한 HBM의 사용 확대

➢ HBM(High Bandwidth Memory)

❖ TSV 기술을 이용해 DRAM 메모리 칩을 수직으로 쌓고, 적층된 칩 사이에 얇은 금속 터널을 만들어 전기적 신호가 칩 간 직접 전달되게 하는 패키징 기술

❖ 데이터 전송 속도의 지연을 최소화하고, 적은 전력으로 많은 양의 데이터를 처리할 수 있음

# 1.4 컴퓨터 구조의 발전 과정



p38~p50

## ■ 슈퍼컴퓨터 구조의 세부적 발전 동향 – 2/2

✓ 전력 효율성 제고 및 냉각 기술 개발

➢ 동적 전압 및 주파수 조정(DVFS), 액체 냉각 기술 등

✓ 클라우드 및 에지 통합(cloud and edge integration)

➢ 슈퍼컴퓨터와 클라우드 서버(cloud server) 결합을 통한 동적 작업량 분할 처리로 컴퓨팅 자원을 증강시키고, 실시간 데이터 처리 작업들을 스마트폰과 같은 단말 장치(edge device)로 이송하여 처리함으로써 슈퍼컴퓨터의 부담을 줄여주는 기술도 개발 중

✓ 양자 컴퓨팅(quantum computing) 기술

➢ 암호 해독, 신약 개발 등과 같은 특정 응용들의 고속 처리에 적합할 것으로 예상되는 양자 컴퓨터가 성공적으로 개발되어 보조프로세서로서의 역할을 추가해준다면, 슈퍼컴퓨터의 역량이 더욱 향상될 것으로 기대됨

# 1.5 컴퓨터 성능

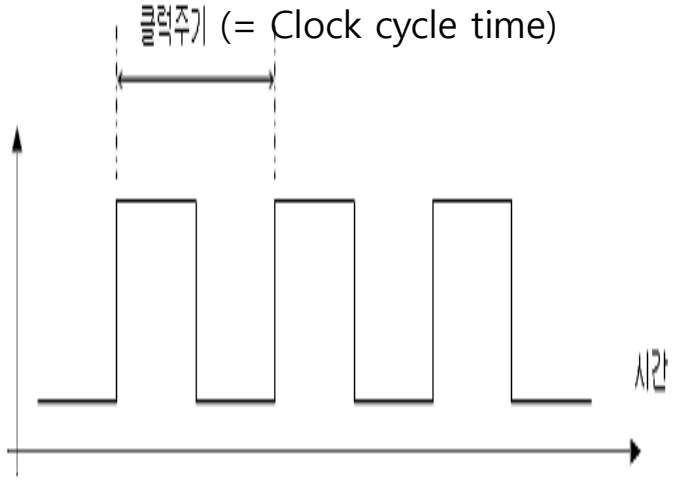


## ■ 컴퓨터 시스템의 성능 측정은 응답시간(즉 실행시간)과 처리량이 중요

- ✓ 처리량 (Throughput)
  - ➢ 일정한 시간 동안 처리하는 작업의 양 → 운영자 입장
- ✓ 응답시간 (Response Time)
  - ➢ 작업 개시에서 종료까지의 시간 → 사용자 입장
- ✓ CPU의 처리속도 뿐만이 아니라 전체 구성(H/W, S/W)의 최적화가 매우 중요

## ■ 컴퓨터의 성능과 실행시간의 관계

- ✓ Performance(성능) = 1 / Execution Time(실행시간)
- ✓ 클럭 속도(주파수) = 1 / 클럭 주기
- ✓ 예) 클럭 속도(주파수) = 200MHz = $200 \times 10^6$Hz
  - ➢ 클럭 주기 = 1/ $200 \times 10^6$Hz = $5 * 10^{-9}$s = 5ns



# 1.5 컴퓨터 성능



## ■ CPI (Clock cycles Per Instruction)

✓ (일반적으로) 명령어마다 실행 시간이 다르므로 모든 명령어의 Clock Cycle에 대한 평균값

## ■ 처리시간 (CPU time)

✓ = 명령어 개수 * CPI * 클럭 사이클 시간(주기)

✓ = (명령어 개수 * CPI) / 클럭 속도(주파수)

## ■ 처리시간의 단축 요인

| | 명령어 수 | CPI | Cycle Time |
|---|---|---|---|
| 프로그램 | ● | | |
| 컴파일러 | ● | (●) | |
| 명령어 Set | ● | ● | |
| Organization | ● | | ● |
| 기술발전 | | | ● |

# 1.5 컴퓨터 성능



## ■ CPU 성능 표시 단위

✓ **MIPS**(Millions of Instructions Per Second, 밉스)

➢ 1초에 수행한 명령어 개수를 100만으로 나눈 값

$$\text{MIPS} = \frac{\text{명령어 개수}}{\text{실행 시간} \times 10^6} = \frac{\text{클록 속도}}{\text{CPI} \times 10^6}$$

MIPS~~와 MFLOP~~는 다른 회사 간의 CPU 또는 같은 회사라도 다른 구조의 CPU 간에는 비교의 의미가 없는 수치이다.
동일 구조의 CPU간에만 사용해야 한다.
MFLOPS는 IEEE754 표준 형태(단정도 32비트, 배정도 64비트)이므로 비교 항목으로 적합하다

✓ **MFLOPS**(Million of FLoating Point Operations Per Second, 메가 플롭스)

➢ 1초에 실행한 부동소수점 연산의 수를 100만으로 나눈 값

$$\text{MFLOPS} = \frac{\text{부동 소수점 연산 개수}}{\text{실행 시간} \times 10^6}$$

# 1.5 컴퓨터 성능



## ■ Benchmark Program 종류

1) Whetstones : 합성 벤치마크

- 부동소수점 연산의 성능을 측정하는 벤치마크 프로그램

2) Dhrystones : 합성 벤치마크

- 정수 연산의 성능을 측정하는 벤치마크 프로그램

3) SPEC(System Performance Evaluation Cooperative)

- 종합적인 성능 측정을 하여 발표하는 비영리 단체
- 아폴로컴퓨터, HP, 밉스컴퓨터시스템스, 선마이크로시스템스 등이 주축(1998년)
- 주로 Workstation급 성능 비교에 많이 사용
- 컴파일러, 부울함수 최소화 프로그램, 스프레드시트, 산술 연산 처리 속도 평가
- C/Unix 에서 사용되는 프로그램에서 선택
- 결과지수 : SPECmark, SPECint, SPECfp

# Thank you

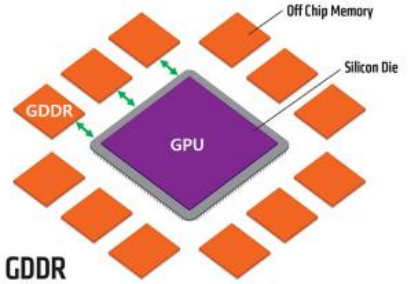
# 참고: DDR, GDDR, HBM 비교

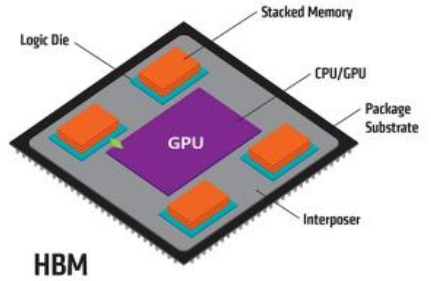
## ■ DDR(Double Data Rate), GDDR(Graphic DDR), HBM(High-Bandwidth Memory)

✓ 모두 DRAM을 기본으로 사용하는 메모리 소자이나 구조, 목적, 특성이 완전히 다름 → 5-1장 상세 학습

| 항목 | DDR (범용 메모리) | GDDR (그래픽 메모리) | HBM (고대역폭 메모리) |
|---|---|---|---|
| 주요 용도 | PC, 서버의 시스템 메모리 (CPU용) | 그래픽카드, 게임 콘솔 (GPU용) | AI 가속기, 슈퍼컴퓨터, 고성능 GPU |
| 최적화 목표 | 낮은 지연 시간 (반응속도) | 높은 대역폭 (처리량) | 초고대역폭 & 전력 효율 |
| 구조 | 2D 평면 구조 (DIMM 모듈) | 2D 평면 구조 (PCB에 직접 실장) | 3D 수직 적층 (TSV 기술) |
| 데이터 버스 폭 | 좁음 (64-bit) | 넓음 (128 ~ 384-bit) | 매우 넓음 (1024-bit 이상) |
| 대역폭 | 낮음 | 높음 | 압도적으로 높음 |
| 전력 소비 | 낮음 | 높음 | 매우 낮음 (대역폭 대비) |
| 물리적 크기 | 큼 (메모리 슬롯 필요) | 중간 (GPU 주변에 배치) | 매우 작음 (칩과 함께 패키징) |
| 장착 방법 | 메인보드 슬롯에 장착 | 기판(PCB)에 직접 납땜 | 프로세서와 함께 단일 패키지로 구성 |
| 가격 | 저렴 | 중간 | 매우 비쌈 |
| 최신 세대 | DDR5 | GDDR6 / GDDR6X / GDDR7 | HBM3 / HBM3E |
| 비유 | 오토바이 퀵서비스 (빠른 배송) | 대형 트럭 (대량 운송) | 자기부상 화물열차 (초고속 대량 운송) |





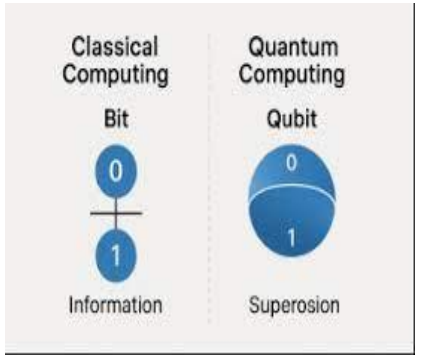



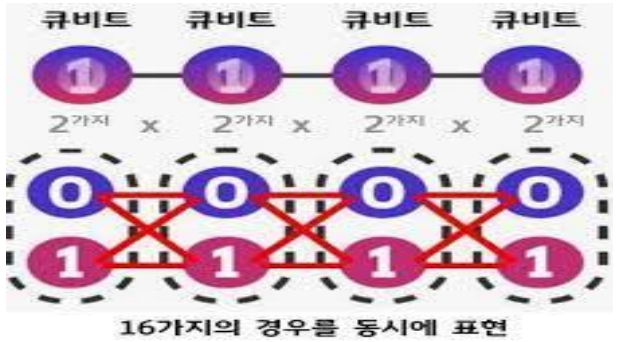
# 참고: 양자 컴퓨터와 일반(이진) 컴퓨터 비교

간단한 개념정도만 이해

## ■ 양자 컴퓨터와 일반(이진) 컴퓨터 비교

- ✓ 다수의 정보를 동시에 연산하는 새로운 개념의 컴퓨터
  - ➢ 양자 컴퓨터에서는 기존의 이진 비트 개념을 혼용해 사용
- ✓ 일반 컴퓨터는 '확실한 상태(0 또는 1)'만을 계산하는 반면, 양자 컴퓨터는 '가능성의 중첩'을 한 번에 계산하여 특정 유형의 문제에서 압도적인 성능을 발휘
  - ➢ 일반 컴퓨터가 외길만 갈 수 있다면, 양자 컴퓨터는 가능한 모든 길을 동시에 탐색하고 정답일 확률이 가장 높은 길을 찾아내는 방식
- ✓ '중첩'과 '얽힘' 등 양자 물리학의 원리를 이용
- ✓ 양자 비트(quantum bit) 인 '큐비트(qubits)'를 이용
  - ❖ 0, 1, 그리고 0과 1의 조합을 동시에 나타내고 저장
  - ❖ 00·01·10·11을 동시에 가짐
- ✓ 성능은 슈퍼컴퓨터 보다 최소 1억 배 이상 나은 결과
- ✓ 머신러닝, 최적화, 신약개발, 암세포 염기서열 분석, 검색 부문의 과제들을 해결 → **특정 분야에만 효과적이며 구현이 매우 힘듦**




| 구분 | 큐비트 (Qubit) | 이진 비트 (Binary Bit) |
|---|---|---|
| 주요 역할 | · 양자 연산 수행 | · 제어, 입출력, 결과 처리 |
| 작동 원리 | · 중첩, 얽힘 등 양자역학적 현상 이용<br>· 0과 1의 상태를 동시에 표현하여 병렬 연산 수행 | · 0 또는 1의 2가지 상태로 정보 표현<br>· 논리 게이트를 이용한 순차적 연산 |
| 담당 업무 | · 양자 알고리즘 실행<br>· 복잡한 문제의 핵심 계산<br>· 신약 개발, 소재 시뮬레이션 등 | · 사용자로부터 명령어 입력<br>· 양자 회로 설계 및 QPU로 전송<br>· 큐비트 상태 제어 및 오류 보정<br>· 연산 결과(큐비트 상태)를 측정하여 '0'과 '1'로 변환 및 저장 |

# 참고: 양자 컴퓨터와 일반(이진) 컴퓨터 비교

간단한 개념정도만 이해

## ■ 양자 컴퓨터와 일반(이진) 컴퓨터 비교

- ✓ 앞으로는 "일상적 계산은 일반 컴퓨터, 난해한 특수 계산은 양자컴퓨터"로 하이브리드 컴퓨팅이 주류가 될 가능성이 높음
- ✓ 일반 컴퓨터 → 범용, 빠른 처리, 안정적인 생태계
- ✓ 양자 컴퓨터 → 특정 초고난도 문제(암호·최적화·신약·물질 시뮬레이션)에 특화

| 구분 | 일반(고전) 컴퓨터 | 양자 컴퓨터 |
|---|---|---|
| 주요 특징 | 전기적 비트(0/1) 기반, 범용적 처리 | 큐비트(중첩·얽힘) 기반, 특수 문제에 강점 |
| 일상적 활용 | 문서 작성, 웹, 게임, 서버 운영 등 | 일상적 활용 불가 (특수 목적 전용) |
| 계산 속도 | 직렬·병렬 연산에 최적화 | 특정 문제에서 지수적 속도 향상 가능 |
| 보안 | 현재 주류 암호체계(RSA, ECC 등) 유지 | 기존 암호 무력화 가능, 양자암호통신(QKD) 연구 |
| 최적화 문제 | 근사 알고리즘·슈퍼컴퓨터 의존 | 물류, 교통, 금융 포트폴리오 최적화에 효과적 |
| 과학 연구 | 기후 모델링, 수치해석, 데이터 시뮬레이션 | 분자·원자 수준 화학·물리 시뮬레이션 (신약, 신소재) |
| AI/머신러닝 | GPU·TPU 기반 딥러닝 | 양자 머신러닝(실험 단계), 패턴 분석 가속 |
| 금융 | 거래 시스템, 리스크 관리(고전적 계산) | 복잡한 리스크·파생상품 모델링 가속 |
| 장점 | 안정성, 범용성, 소프트웨어 생태계 완성 | 특정 문제에서 압도적 계산 성능 |
| 한계 | 초고난도 조합 최적화에 비효율 | 오류율 높음, 대규모 큐비트 확보 어려움 |